embarca tech | Residência Tecnológica em Sistemas Embarcados

Modulação por Largura de Pulso (PWM)

Unidade 4 | Capítulo 7 – Aula Síncrona (06/02/2025)

Prof. Dr Ricardo Menezes Prates

Executores:
INSTITUTO FEDERAL Piauí
INSTITUTO FEDERAL Rio Grande do Norte
INSTITUTO FEDERAL Maranhão
INSTITUTO FEDERAL Ceará
cepedi
INSTITUTO HARDWARE BR
Coordenação: Softex
Iniciativa: MINISTÉRIO DA CIÊNCIA, TECNOLOGIA E INOVAÇÃO
GOVERNO FEDERAL BRASIL UNIÃO E RECONSTRUÇÃO

- Introdução
  - Planejamento da aula
- *Modulação por Largura de Pulso (PWM)*
  - *Módulo PWM do* RP2040
- *Exemplos de aplicação*
  - *Configuração do módulo PWM*
  - Rotina PWM com alteração do Ciclo de Trabalho (*Duty Cycle*)
  - Rotina PWM com interrupção vinculada ao período (*Wrap*)
  - Atividade – Controle de Servomotor por PWM
- Conclusões

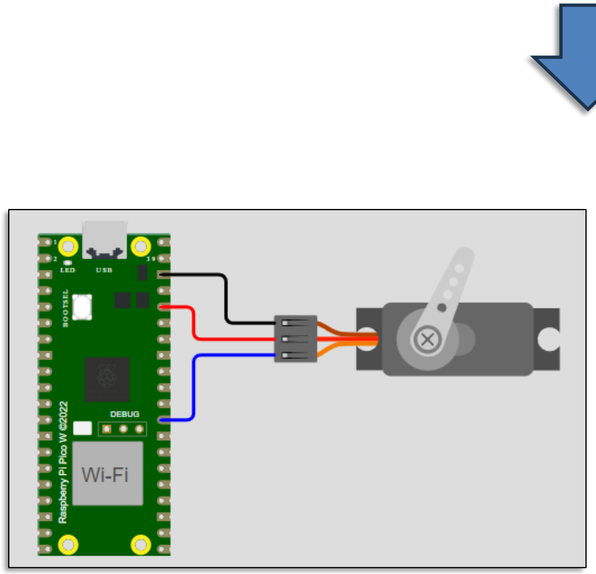
Introdução

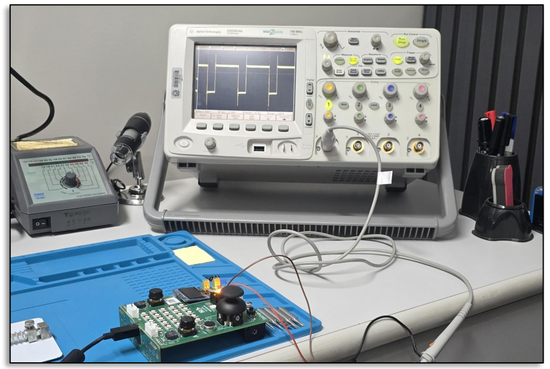
Objetivos

- Apresentar ao discente os conceitos básicos da **Modulação por Largura de Pulso**, e do periférico PWM presente no microcontrolador RP2040.

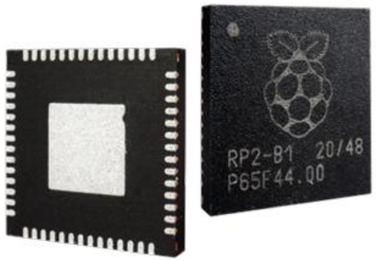
- Implementar diferentes exemplos de ajuste do **Ciclo de Trabalho** (*Duty Cycle*) para o controle da Tensão média e da Potência disponibilizada para periféricos externos.

Código fonte





Planejamento da Aula



- Pré-requisitos para esta aula:

✓ C Básico e Pico SDK- RP2040;

✓ Portas de Entrada e Saída (GPIO);

✓ Rotinas de *delay* básico;

✓ Rotinas de interrupção;

✓ *Clock* e Temporizadores.

WOKWI

raspberrypi/**pico-sdk**

Planejamento da Aula

- O que ainda está por vir:

✓ Conversor analógico digital;

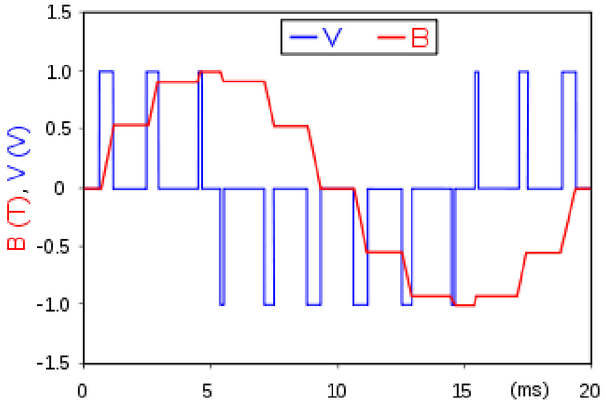
"A mente que se abre a uma nova ideia jamais voltará ao seu tamanho original." - frase atribuída a Albert Einstein.

Modulação por Largura de Pulso (PWM)

PWM

- Características:

✓ A **modulação por Largura de Pulsos** (em inglês, *Pulse Width Modulation* - PWM) é uma técnica que envolve a variação da largura dos pulsos em um ciclo do sinal elétrico.



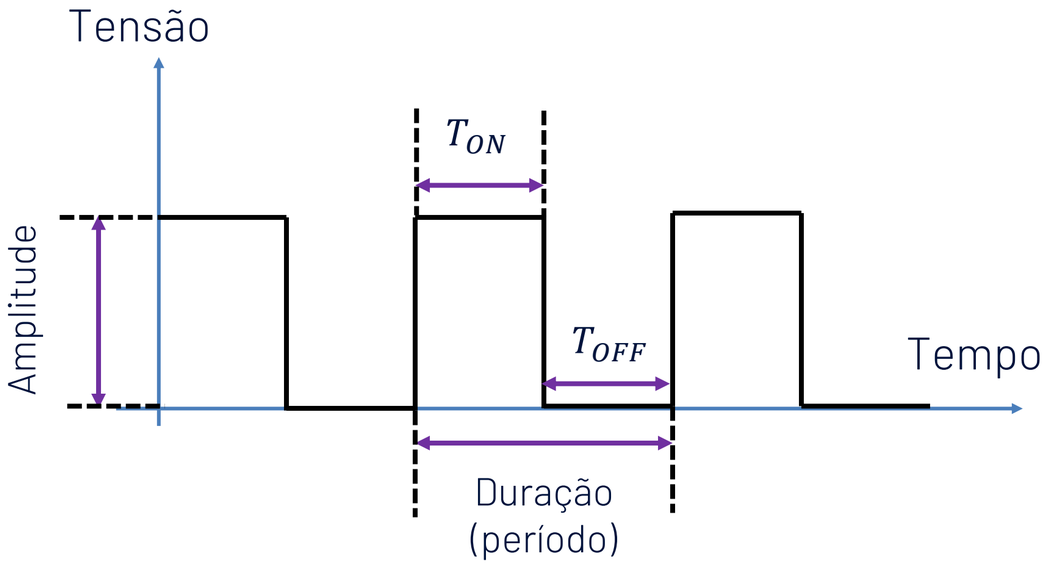
PWM

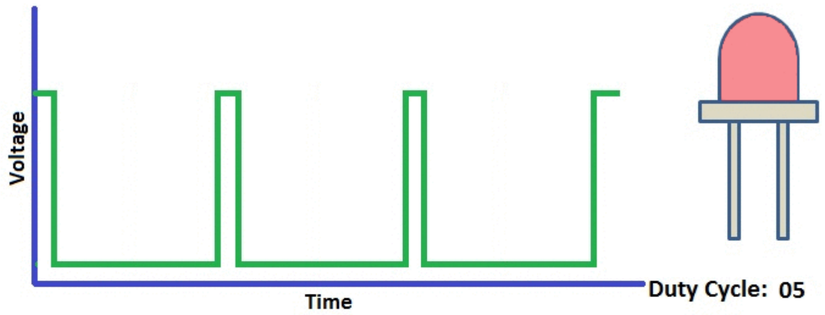
- Características:

✓ A **modulação por Largura de Pulsos** (em inglês, *Pulse Width Modulation* - PWM) é uma técnica que envolve a variação da largura dos pulsos em um ciclo do sinal elétrico.

✓ O **sinal PWM** é composto por uma série de pulsos retangulares com largura variável.

✓ É realizado o ajuste do tempo que o sinal fica em **nível alto** em cada ciclo para controlar a tensão média e a potência fornecida.





Fonte: www.citisystems.com.br/pwm/

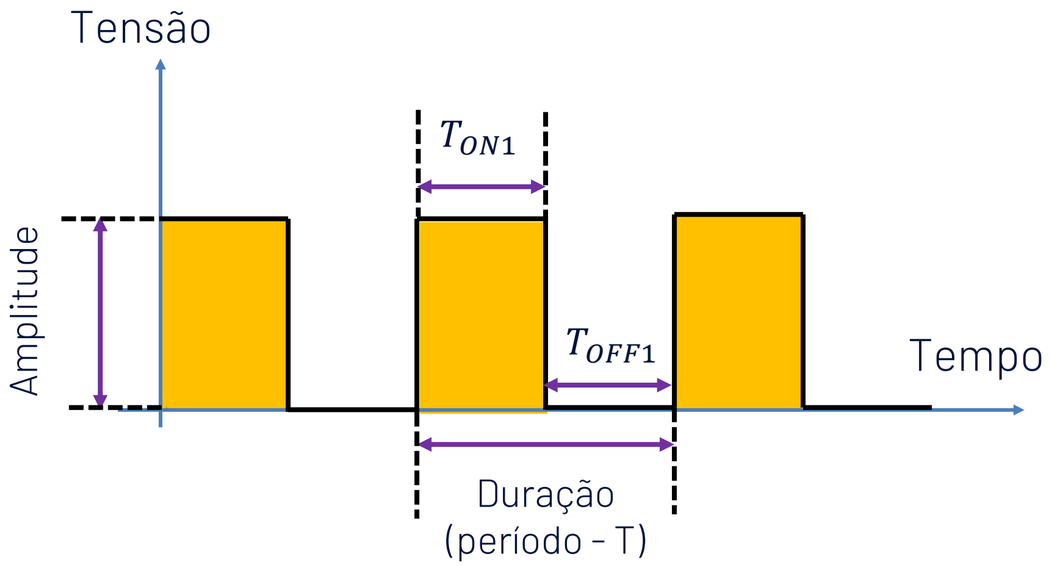
PWM

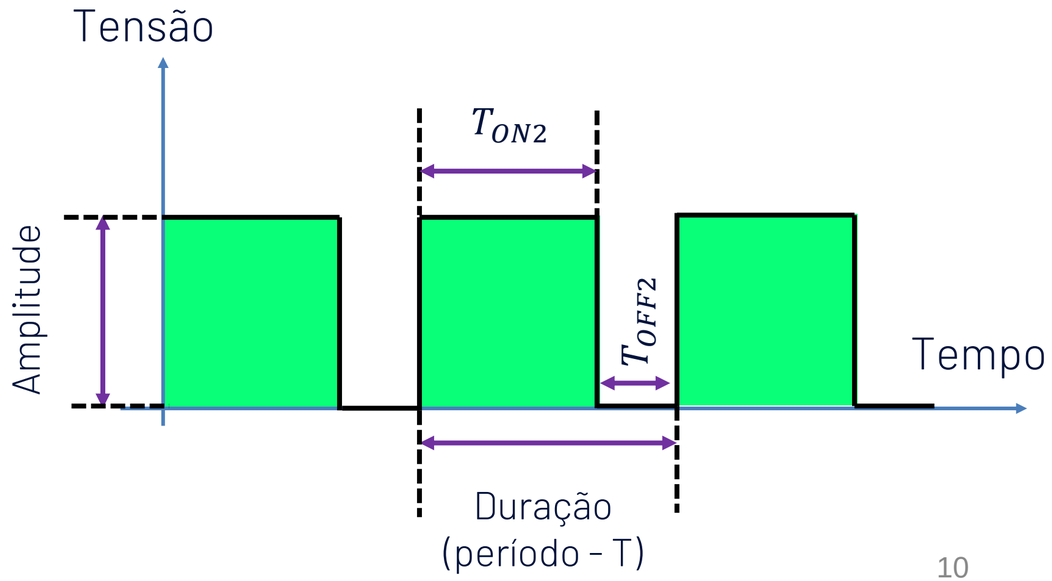
- Características:

✓ A **modulação por Largura de Pulsos** (em inglês, *Pulse Width Modulation* - PWM) é uma técnica que envolve a variação da largura dos pulsos em um ciclo do sinal elétrico.

✓ O **sinal PWM** é composto por uma série de pulsos retangulares com largura variável.

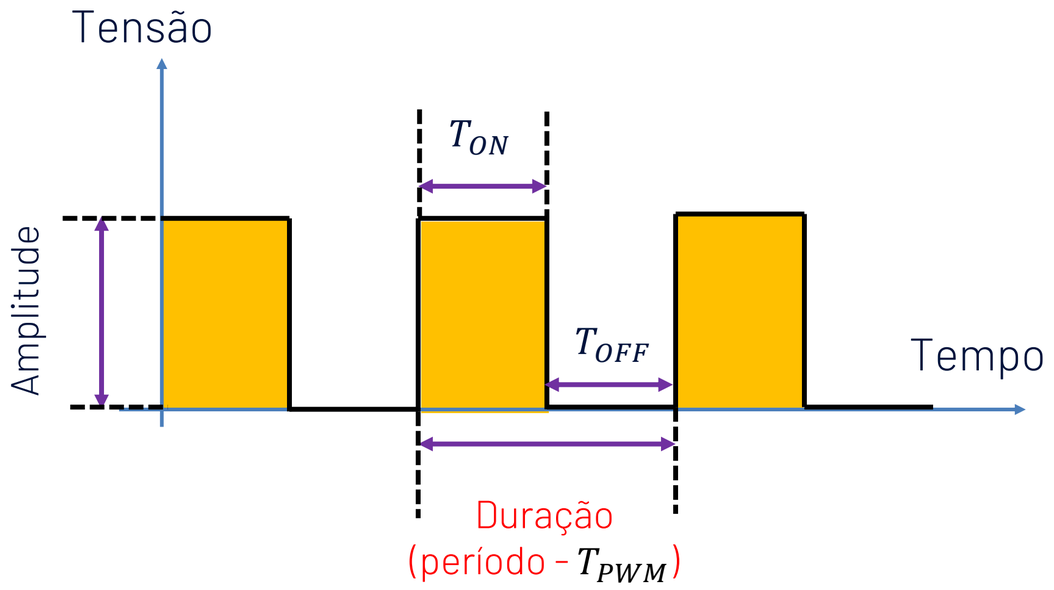
✓ É realizado o ajuste do tempo que o sinal fica em **nível alto** em cada ciclo para controlar a tensão média e a potência fornecida.





PWM

- Revisão da aula sobre Temporizadores:



✓ O **período** $(\boldsymbol{T})$ de um sinal periódico é o tempo entre duas bordas de subida ou de descida;

$$T_{PWM} = T_{ON} + T_{OFF}$$

✓ A **frequência** $(\boldsymbol{f})$ é a taxa com que ela se repete por segundo e é medida em Hertz (Hz).

$$\boxed{f_{PWM} = \frac{1}{T_{PWM}}}$$

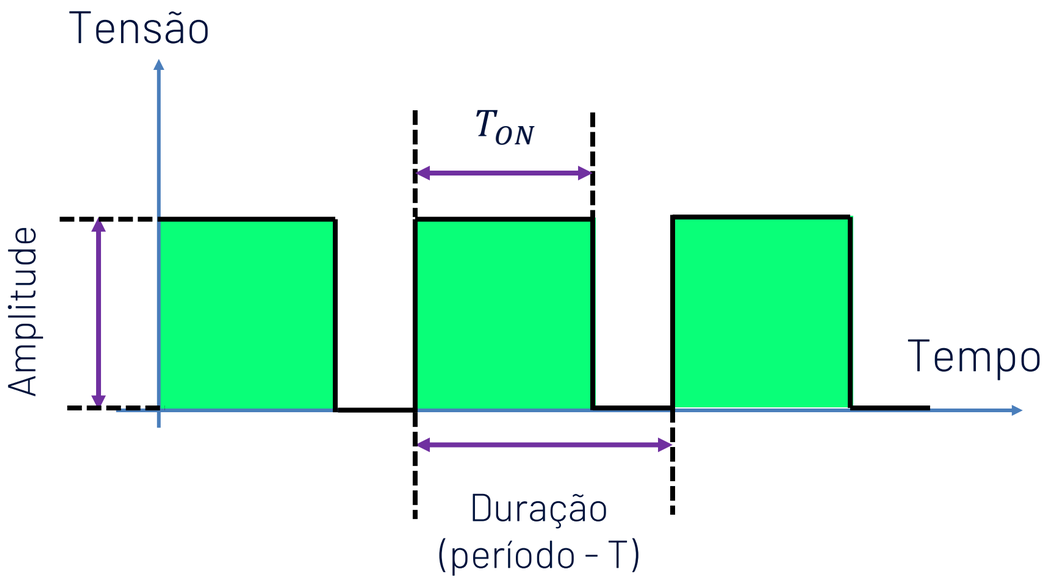
PWM

- Como funciona?

✓ O tempo que o sinal está em nível alto (T_{ON}) , podendo ser considerado como um período de "chave ligada", chamamos de largura pulso ou ciclo ativo;

✓ **Quanto mais longo o pulso**, maior é a tensão média e maior a quantidade de potência transferida para a carga.

✓ O ciclo de trabalho (em inglês, ***Duty Cycle*** – D_C) é a relação da largura do pulso sobre o período, ou seja, o tempo em que o sinal está em nível alto (ligado) e o tempo em que está em nível baixo (desligado).



$$D_C = \left(\frac{T_{ON}}{T_{PWM}}\right) \times 100$$

$$D_C = \left(\frac{T_{ON}}{T_{ON} + T_{OFF}}\right) \times 100$$

$D_C \rightarrow$ Valor em porcentagem %

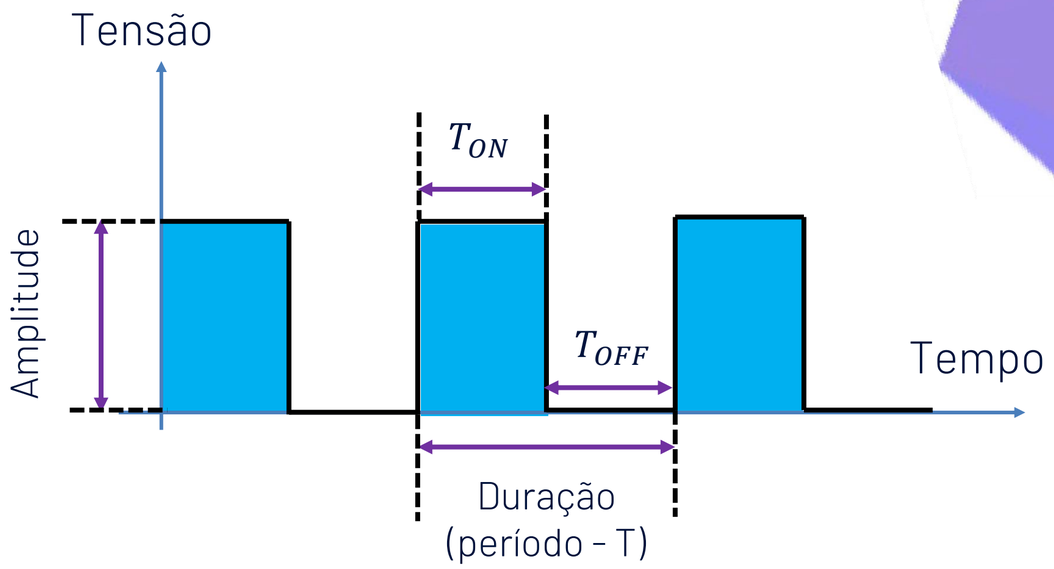
PWM

- *Ciclo de Trabalho (Duty Cycle):*

✓ O ciclo de trabalho (em inglês, ***Duty Cycle*** – D_C) é a relação da largura do pulso sobre o período, ou seja, o tempo em que o sinal está em nível alto (ligado) e o tempo em que está em nível baixo (desligado).

Exemplo: Qual o Duty Cycle de um sinal PWM com as seguintes especificações:

1. Frequência (f): 1 kHz;
2. Período (T): 1 ms
3. Tempo de pulso (ciclo ativo): 0,5 ms.



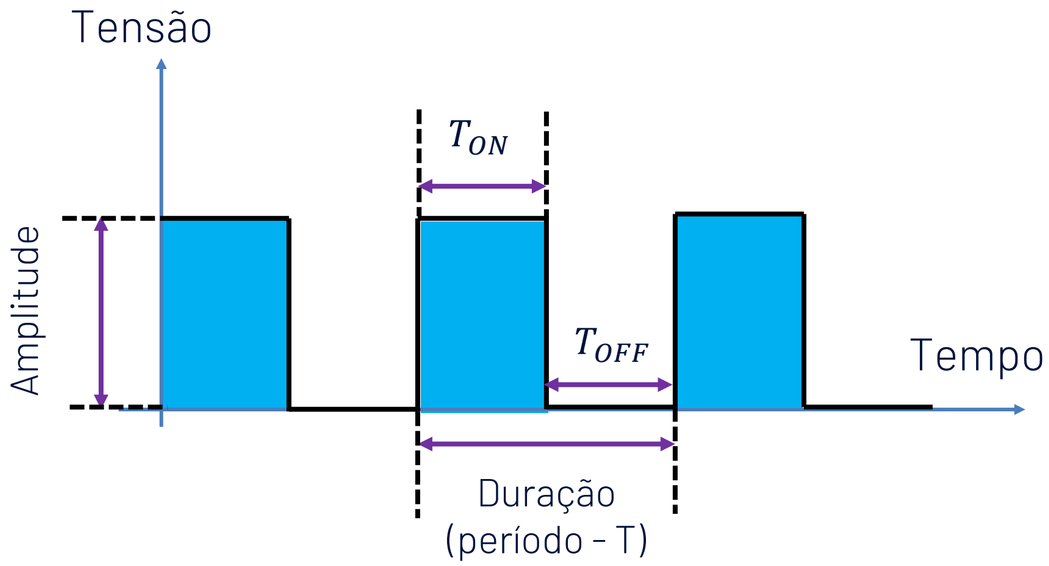
$$D_C = \left(\frac{T_{ON}}{T_{PWM}}\right) \times 100$$

$$D_C = \left(\frac{0{,}5 \times 10^{-3}}{1 \times 10^{-3}}\right) \times 100$$

$$\boxed{D_C = 50\%}$$

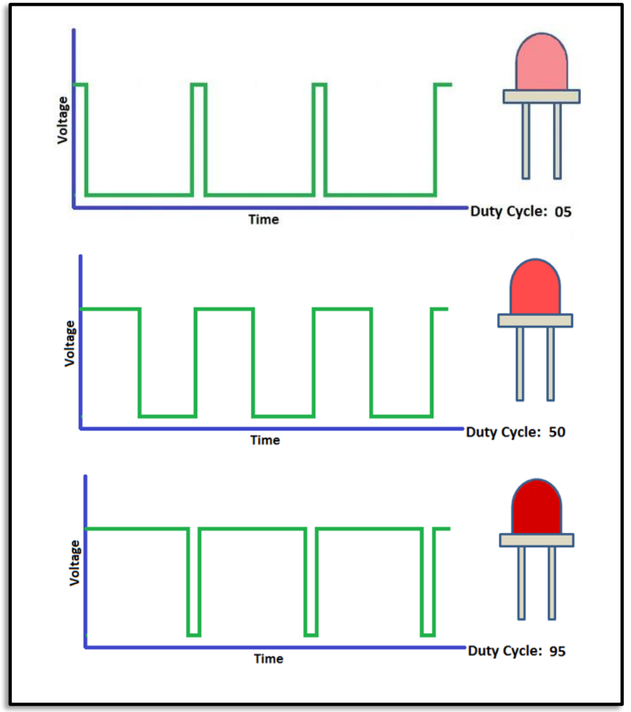
PWM

- *Tensão média (V_{med}):*



✓ A **tensão média (V_{med})** é calculada com base na tensão máxima do sinal PWM (Vpico) e no ciclo de trabalho (Duty Cycle). Ela está relacionada a energia que é transferida para a carga durante o ciclo de trabalho do sinal PWM.

$$V_{med} = V_{pico} \times D_C$$



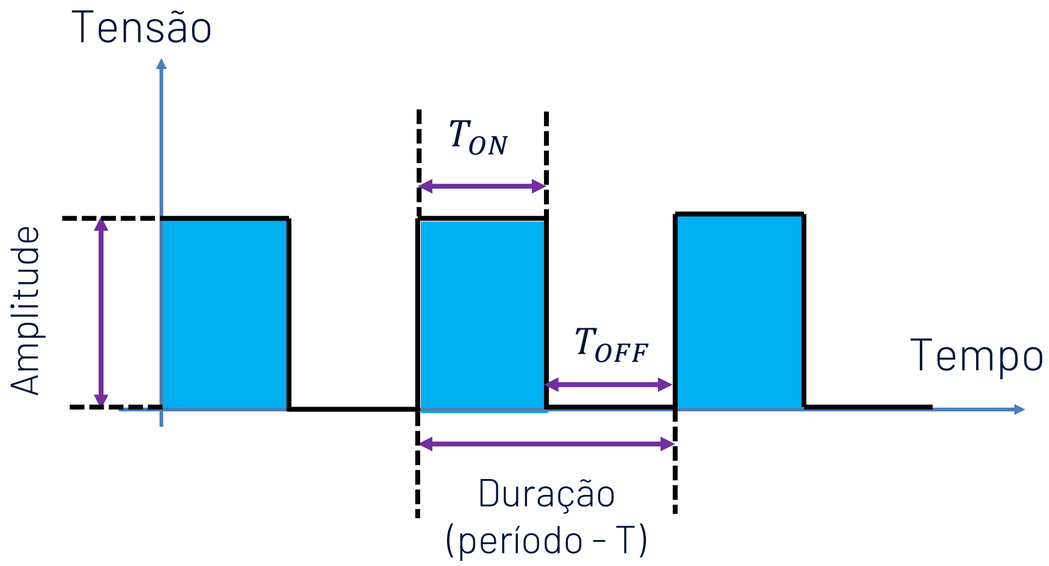
PWM

- *Tensão média (V_{med}):*

✓ A **tensão média (V_{med})** é calculada com base na tensão máxima do sinal PWM (Vpico) e no ciclo de trabalho (Duty Cycle). Ela está relacionada a energia que é transferida para a carga durante o ciclo de trabalho do sinal PWM.

Exemplo: Qual Tensão Média de um sinal PWM com as seguintes especificações:

1. Tensão de Pico (V_{pico}): 3,3 V;
2. Duty Cycle (D_C): 50% (ou 0,5).



$$V_{med} = V_{pico} \times D_C$$

$$V_{med} = 3{,}3 \times 0{,}5$$

$$\boxed{V_{med} = 1{,}65\ \text{V}}$$

PWM

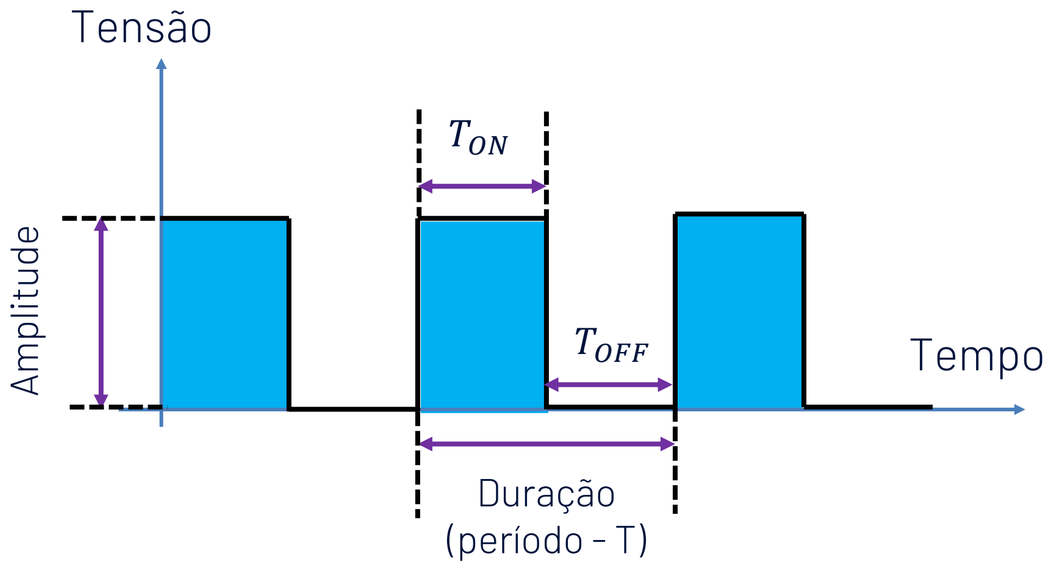
- *Tensão eficaz (V_{RMS}):*

✓ A **tensão média (V_{RMS})** é calculada com base na tensão máxima do sinal PWM (Vpico) e no ciclo de trabalho (Duty Cycle).

✓ O V_{RMS} de um sinal é um valor constante (contínuo) que seria capaz de produzir sobre uma resistência **a mesma potência elétrica que o sinal original produziria sobre aquela mesma resistência**.

✓ Para o exemplo anterior:

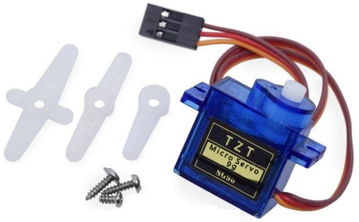
$$V_{RMS} = 3{,}3 \times \sqrt{0{,}5} = 2{,}33\ \mathrm{V}$$

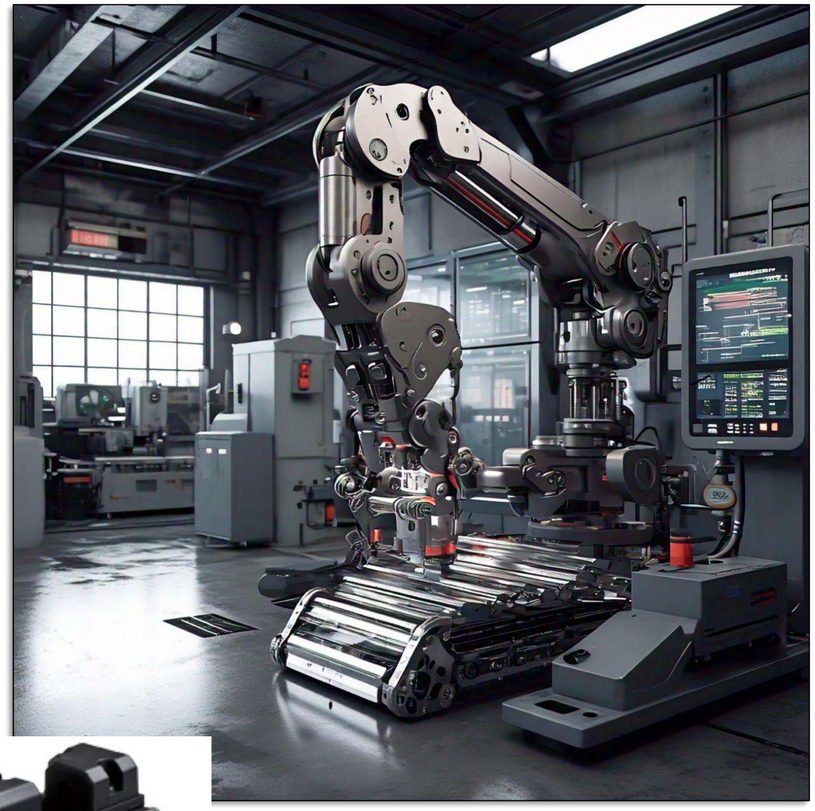


$$V_{RMS} = V_{pico} \times \sqrt{D_C}$$

PWM

- Exemplos de aplicação de PWM:
  - Controle de Luminosidade de LEDs;
  - Controle de posição de servomotor;
  - Geração de sinais de áudio;
  - Controle de motores CC;
  - Geração de sinal analógico;
  - Controlador de carga;
  - Comunicação de dados.





Módulo PWM do RP2040

PWM – RP2040

- Características do *PWM (4.5)*:

✓ O RP2040 conta com um periférico interno para Modulação por Largura de Pulso (PWM). ①

O módulo PWM apresenta:

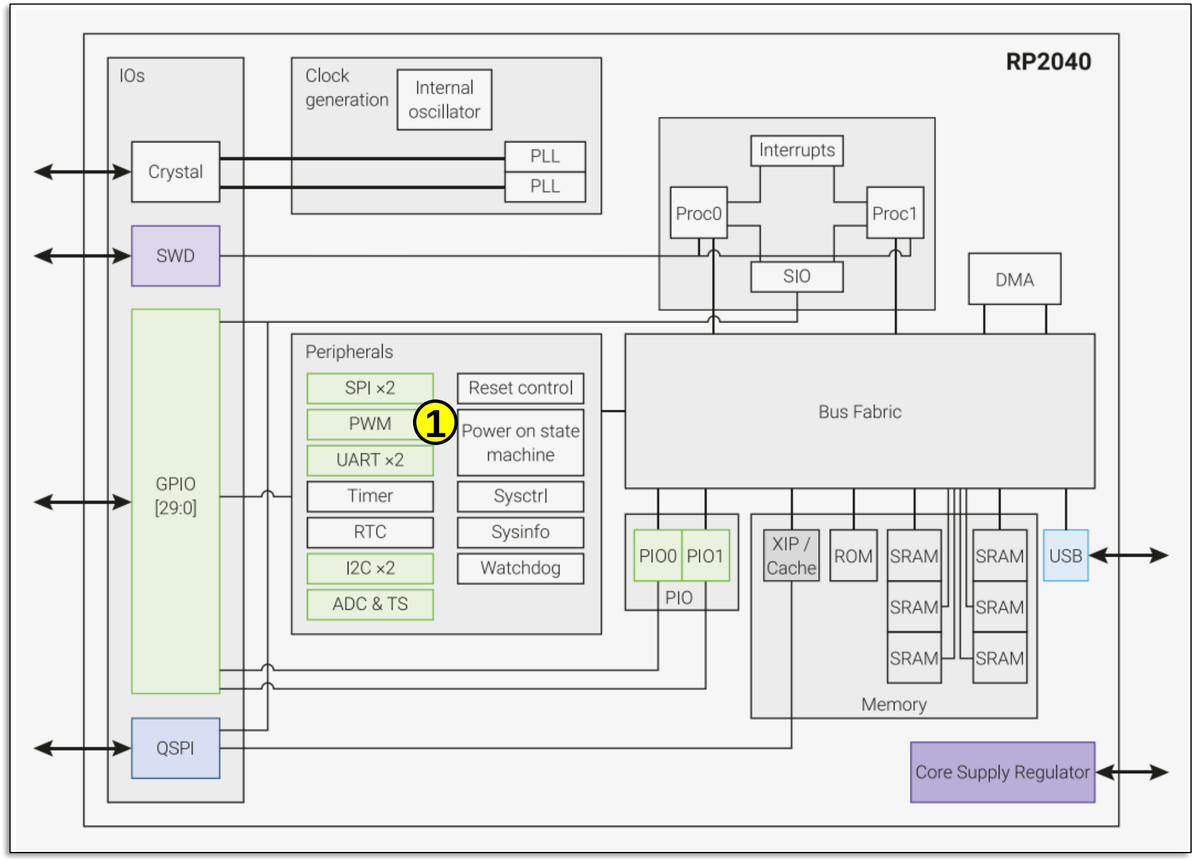
1) 08 geradores de PWM (*slices*) idênticos, cada um capaz de gerar 02 sinais de saída independentes;

2) Total de 16 saídas PWM controláveis.

3) Os 30 pinos GIO podem ser controlados pelo bloco PWM.

4) Os *slices* podem ser configurados para medir o período ou ciclo ativo de sinais de entrada.

Arquitetura interna do microcontrolador rp2040.



PWM – RP2040

- Todos as 30 GPIOs podem ser utilizadas no RP2040 (4.5.1):

✓ Qualquer pino GPIO pode ser configurado como uma saída PWM.

GPIO	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
PWM Channel	0A	0B	1A	1B	2A	2B	3A	3B	4A	4B	5A	5B	6A	6B	7A	7B
GPIO	16	17	18	19	20	21	22	23	24	25	26	27	28	29		
PWM Channel	0A	0B	1A	1B	2A	2B	3A	3B	4A	4B	5A	5B	6A	6B		

PWM – RP2040

- Representação de um único *slice* (4.5.1):

✓ Contador de 16 bits;

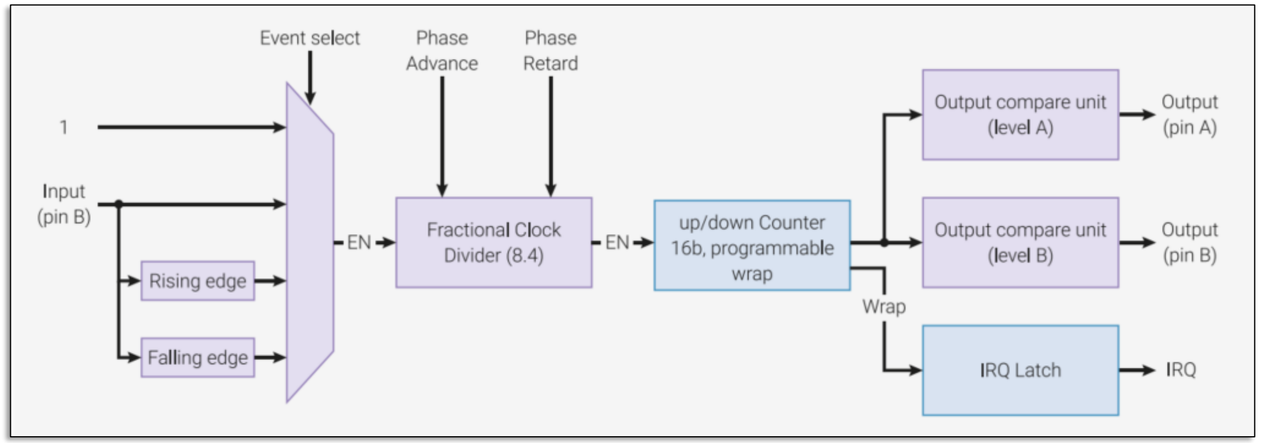
✓ 8.4 - Divisor de clock fracional (DIV);

✓ Dois canais indepentes – D_C de 0% a 100%;

✓ Wrap (TOP) – valor máximo de contagem configurável, com 16 bits (65.535).

✓ Requisição de interrupções (IRQ).

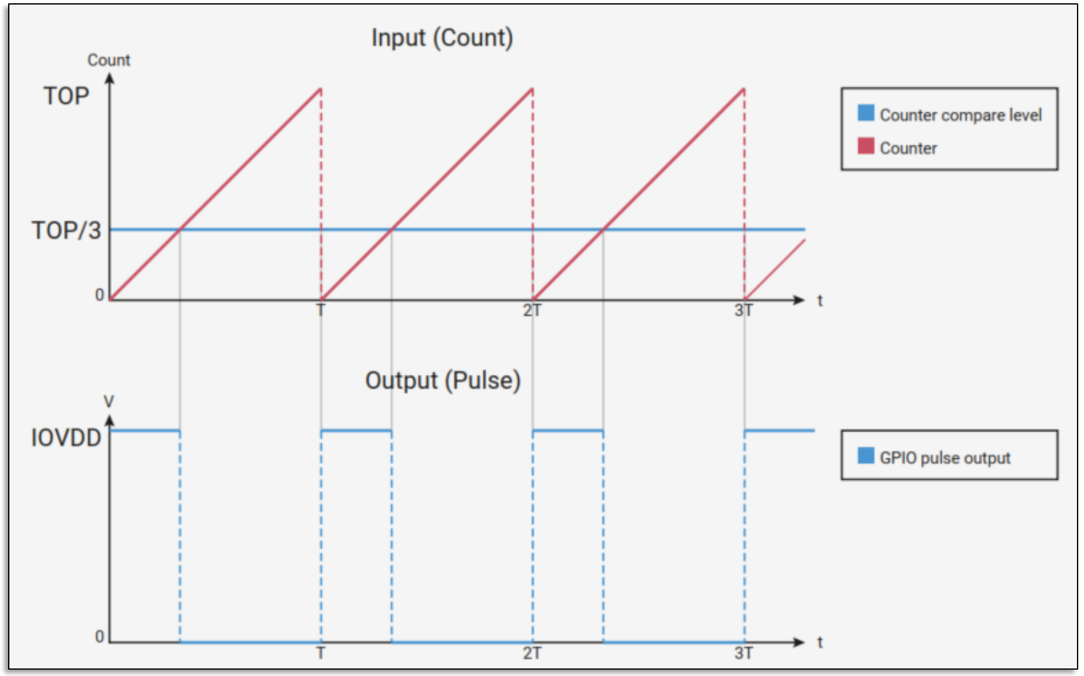
Diagrama de blocos de um *slice*:



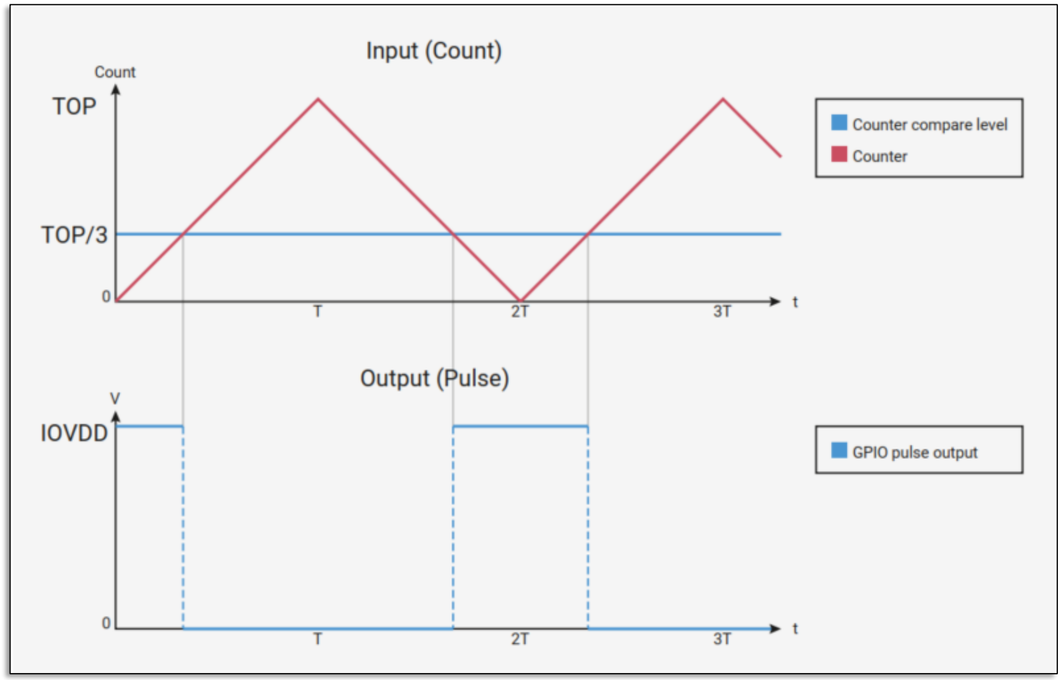
IRQ	Interrupt Source	IRQ	Interrupt Source	IRQ	Interrupt Source	IRQ	Interrupt Source	IRQ	Interrupt Source
0	TIMER_IRQ_0	6	XIP_IRQ	12	DMA_IRQ_1	18	SPI0_IRQ	24	I2C1_IRQ
1	TIMER_IRQ_1	7	PIO0_IRQ_0	13	IO_IRQ_BANK0	19	SPI1_IRQ	25	RTC_IRQ
2	TIMER_IRQ_2	8	PIO0_IRQ_1	14	IO_IRQ_QSPI	20	UART0_IRQ		
3	TIMER_IRQ_3	9	PIO1_IRQ_0	15	SIO_IRQ_PROC0	21	UART1_IRQ		
4	PWM_IRQ_WRAP	10	PIO1_IRQ_1	16	SIO_IRQ_PROC1	22	ADC_IRQ_FIFO		
5	USBCTRL_IRQ	11	DMA_IRQ_0	17	CLOCKS_IRQ	23	I2C0_IRQ		

PWM – RP2040

- Modulação por Largura de Pulso (4.5.1) – Figura 104:



Comportamento padrão.



Modo com correção de fase – não é função do *Duty Cycle* - *CSR_PH_CORRECT =1* .

PWM – RP2040

- Configuração da frequência PWM (4.5.2.6):

✓ É controlada por 03 parâmetros: TOP, DIV e *CSR_PH_CORRECT*.

minimum speed of one count per $255\frac{15}{16}$ cycles. The period in clock cycles can be calculated as:

$$\text{period} = (\text{TOP} + 1) \times (\text{CSR_PH_CORRECT} + 1) \times \left(\text{DIV_INT} + \frac{\text{DIV_FRAC}}{16}\right)$$

The output frequency can then be determined based on the system clock frequency:

$$f_{PWM} = \frac{f_{sys}}{\text{period}} = \frac{f_{sys}}{(\text{TOP} + 1) \times (\text{CSR_PH_CORRECT} + 1) \times \left(\text{DIV_INT} + \frac{\text{DIV_FRAC}}{16}\right)}$$

Vamos usar:

$$f_{PWM} = \frac{f_{clock}}{\left(d_i + \frac{d_f}{16}\right) \times (\text{wrap} + 1)}$$

d_i -> Divisor inteiro

d_f -> Divisor fracionário

Obs.: frequência mínima de 7,5Hz para um clock de sistema de 125MHz.

Temporizadores - RP2040

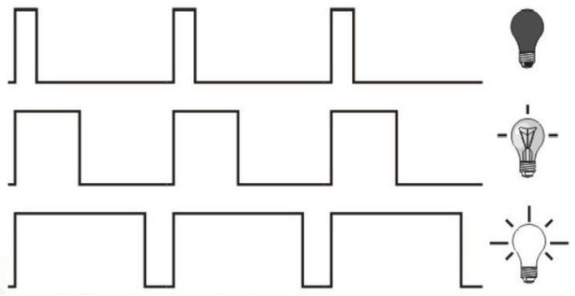
- No contexto desta aula, faremos seguintes abordagens:

✓ Configuração do módulo PWM;

✓ Controle do Duty Cycle;

✓ PWM com Interrupção;

✓ **Atividade** – controle de servomotor.

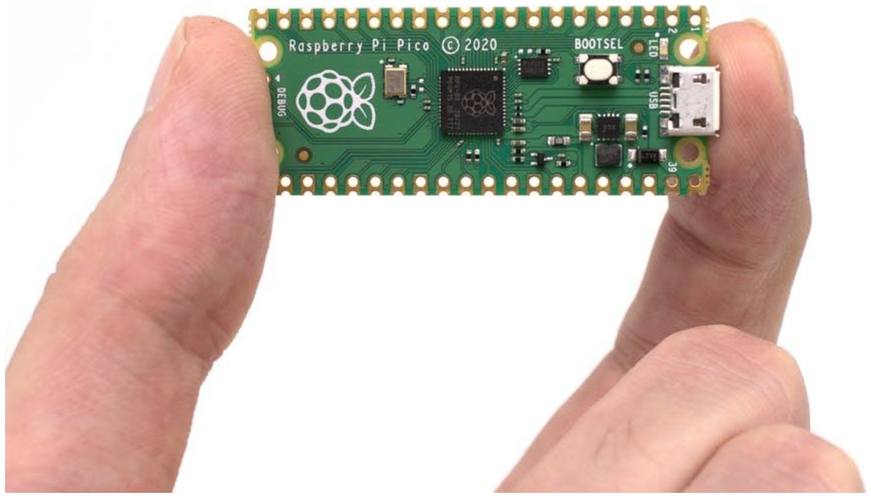


Material complementar

Exemplos de aplicação

Vamos para o microcontrolador!

Exemplo 1



Configuração do módulo PWM

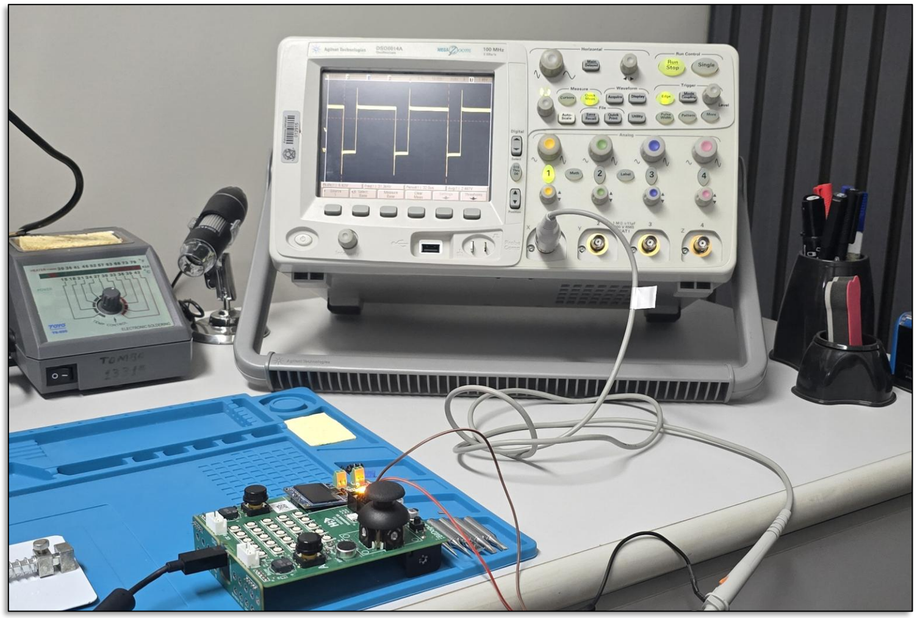
Lembrete: Organização básica do código fonte.

```
1    #include <stdio.h>
2    #include "pico/stdlib.h" //biblioteca SDK padrão do PICO - funcionalidades para programação básica.
3
4    // Definição funções, macros ou variáveis/constantes globais (1)
5
6    void main() // função principal
7    {
8         // inicializações de hardware e software (2)
9
10        while(true) // loop infinito
11        {
12             /* tarefas a serem executadas pelo microcontrolador durante o funcionamento */ (3)
13        }
14   }
```

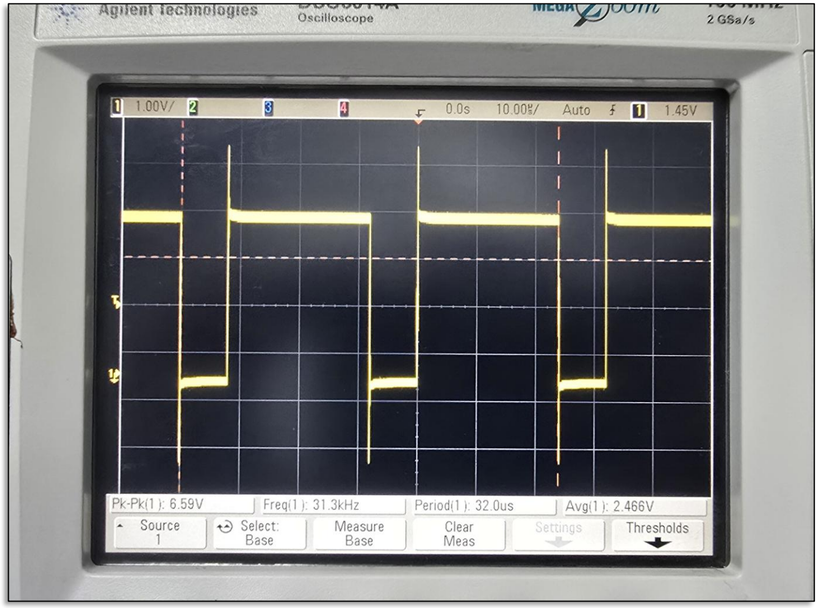
Configuração do módulo PWM – Exemplo 1

Objetivo: Realizar o configuração do módulo PWM da GPIO 12.

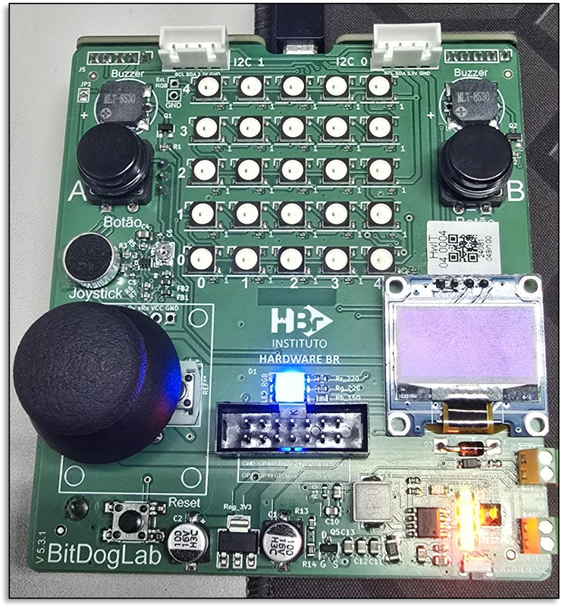
Experimento



Osciloscópio



BitDogLab



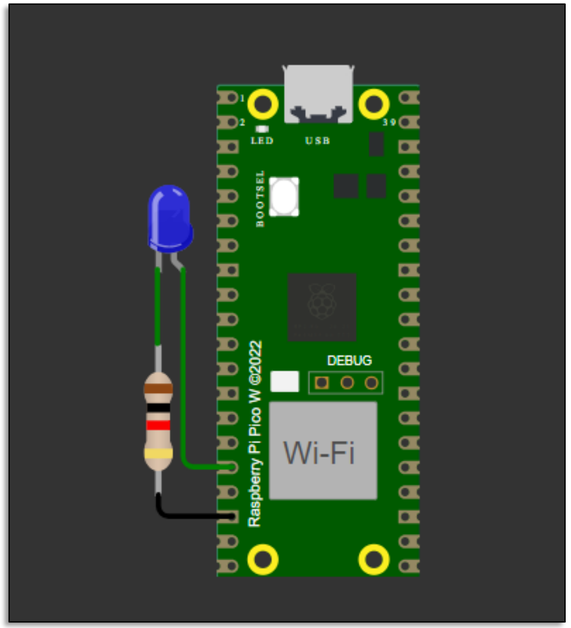
Configuração do módulo PWM – Exemplo 1

Código fonte.

```c
#include <stdio.h> //biblioteca padrão da linguagem C
#include "pico/stdlib.h" //subconjunto central de bibliotecas do SDK Pico
#include "hardware/pwm.h" //biblioteca para controlar o hardware de PWM

#define PWM_LED 12 //pino do LED conectado a GPIO como PWM

//rotina principal
int main()
{
    stdio_init_all(); //inicializa o sistema padrão de I/O

    gpio_set_function(PWM_LED, GPIO_FUNC_PWM); //habilitar o pino GPIO como PWM

    uint slice = pwm_gpio_to_slice_num(PWM_LED); //obter o canal (slice) PWM da GPIO

    pwm_set_clkdiv(slice, 2.0); //define o divisor de clock do PWM

    pwm_set_wrap(slice, 2000); //definir o valor de wrap – valor máximo do contador PWM

    pwm_set_gpio_level(PWM_LED, 1500); //definir o ciclo de trabalho (duty cycle) do pwm

    pwm_set_enabled(slice, true); //habilitar o pwm no slice correspondente

    //loop principal
    while (true) {

        printf("slice:%d\n",slice);
        sleep_ms(1000);
    }
}
```

Cod: pwm_config_01

Simulação Wokwi.



GitHub

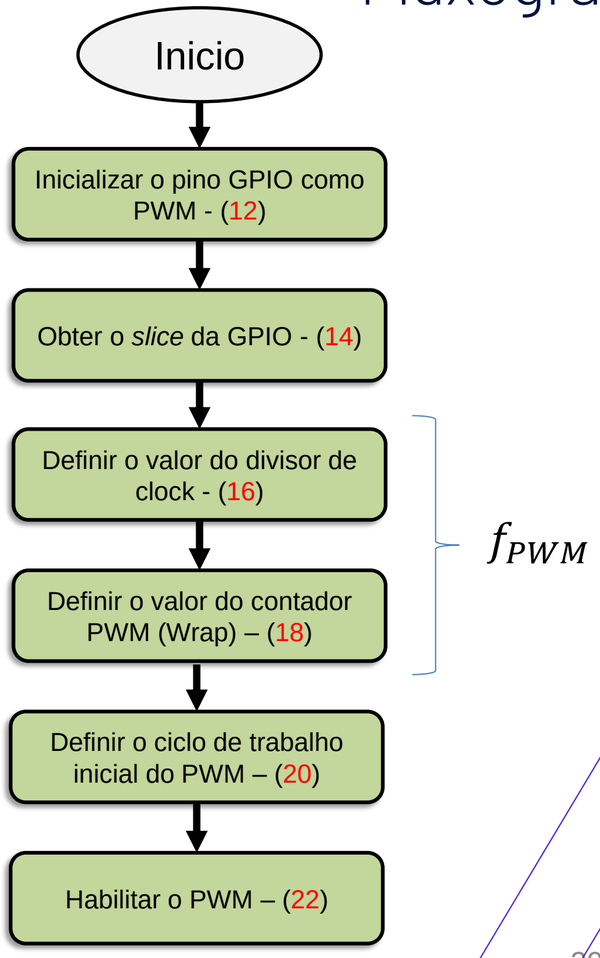
Configuração do módulo PWM – Exemplo 1

Código fonte.

```c
1   #include <stdio.h> //biblioteca padrão da linguagem C
2   #include "pico/stdlib.h" //subconjunto central de bibliotecas do SDK Pico
3   #include "hardware/pwm.h" //biblioteca para controlar o hardware de PWM
4
5   #define PWM_LED 12 //pino do LED conectado a GPIO como PWM
6
7   //rotina principal
8   int main()
9   {
10      stdio_init_all(); //inicializa o sistema padrão de I/O
11
12      gpio_set_function(PWM_LED, GPIO_FUNC_PWM); //habilitar o pino GPIO como PWM
13
14      uint slice = pwm_gpio_to_slice_num(PWM_LED); //obter o canal (slice) PWM da GPIO
15
16      pwm_set_clkdiv(slice, 2.0); //define o divisor de clock do PWM
17
18      pwm_set_wrap(slice, 2000); //definir o valor de wrap – valor máximo do contador PWM
19
20      pwm_set_gpio_level(PWM_LED, 1500); //definir o ciclo de trabalho (duty cycle) do pwm
21
22      pwm_set_enabled(slice, true); //habilitar o pwm no slice correspondente
23
24      //loop principal
25      while (true) {
26
27          printf("slice:%d\n",slice);
28          sleep_ms(1000);
29      }
30  }
```

Cod: pwm_01

Fluxograma



Configuração do módulo PWM – Exemplo 1

Código fonte.

```c
#include <stdio.h> //biblioteca padrão da linguagem C
#include "pico/stdlib.h" //subconjunto central de bibliotecas do SDK Pico
#include "hardware/pwm.h" //biblioteca para controlar o hardware de PWM

#define PWM_LED 12 //pino do LED conectado a GPIO como PWM

//rotina principal
int main()
{
    stdio_init_all(); //inicializa o sistema padrão de I/O

    gpio_set_function(PWM_LED, GPIO_FUNC_PWM); //habilitar o pino GPIO como PWM

    uint slice = pwm_gpio_to_slice_num(PWM_LED); //obter o canal (slice) PWM da GPIO

    pwm_set_clkdiv(slice, 2.0); //define o divisor de clock do PWM

    pwm_set_wrap(slice, 2000); //definir o valor de wrap – valor máximo do contador PWM

    pwm_set_gpio_level(PWM_LED, 1500); //definir o ciclo de trabalho (duty cycle) do pwm

    pwm_set_enabled(slice, true); //habilitar o pwm no slice correspondente

    //loop principal
    while (true) {

        printf("slice:%d\n",slice);
        sleep_ms(1000);
    }
}
```

Cod: pwm_01

Frequência de PWM.

$$f_{PWM} = \frac{f_{clock}}{\left(d_i + \frac{d_f}{16}\right) \times (\text{wrap} + 1)}$$

d_i -> Divisor inteiro

d_f -> Divisor fracionário

Configuração do módulo PWM – Exemplo 1

Código fonte.

```c
#include <stdio.h> //biblioteca padrão da linguagem C
#include "pico/stdlib.h" //subconjunto central de bibliotecas do SDK Pico
#include "hardware/pwm.h" //biblioteca para controlar o hardware de PWM

#define PWM_LED 12 //pino do LED conectado a GPIO como PWM

//rotina principal
int main()
{
    stdio_init_all(); //inicializa o sistema padrão de I/O

    gpio_set_function(PWM_LED, GPIO_FUNC_PWM); //habilitar o pino GPIO como PWM

    uint slice = pwm_gpio_to_slice_num(PWM_LED); //obter o canal (slice) PWM da GPIO

    pwm_set_clkdiv(slice, 2.0); //define o divisor de clock do PWM

    pwm_set_wrap(slice, 2000); //definir o valor de wrap – valor máximo do contador PWM

    pwm_set_gpio_level(PWM_LED, 1500); //definir o ciclo de trabalho (duty cycle) do pwm

    pwm_set_enabled(slice, true); //habilitar o pwm no slice correspondente

    //loop principal
    while (true) {

        printf("slice:%d\n",slice);
        sleep_ms(1000);
    }
}
```

Cod: pwm_01

Frequência de PWM.

$$f_{PWM} = \frac{125 \times 10^6}{(2 + \frac{0}{16}) \times 2 \times 10^3}$$

$$f_{PWM} = 31.250 \text{ Hz}$$

Configuração do módulo PWM – Exemplo 1

Código fonte.

```c
#include <stdio.h> //biblioteca padrão da linguagem C
#include "pico/stdlib.h" //subconjunto central de bibliotecas do SDK Pico
#include "hardware/pwm.h" //biblioteca para controlar o hardware de PWM

#define PWM_LED 12 //pino do LED conectado a GPIO como PWM

//rotina principal
int main()
{
    stdio_init_all(); //inicializa o sistema padrão de I/O

    gpio_set_function(PWM_LED, GPIO_FUNC_PWM); //habilitar o pino GPIO como PWM

    uint slice = pwm_gpio_to_slice_num(PWM_LED); //obter o canal (slice) PWM da GPIO

    pwm_set_clkdiv(slice, 2.0); //define o divisor de clock do PWM

    pwm_set_wrap(slice, 2000); //definir o valor de wrap – valor máximo do contador PWM

    pwm_set_gpio_level(PWM_LED, 1500); //definir o ciclo de trabalho (duty cycle) do pwm

    pwm_set_enabled(slice, true); //habilitar o pwm no slice correspondente

    //loop principal
    while (true) {

        printf("slice:%d\n",slice);
        sleep_ms(1000);
    }
}
```

Cod: pwm_01

Frequência de PWM.

$$f_{PWM} = 31.250 \text{ Hz}$$

$$T_{PWM} = \frac{1}{f_{PWM}} = 32\mu s$$

Configuração do módulo PWM – Exemplo 1

Código fonte.

```c
#include <stdio.h> //biblioteca padrão da linguagem C
#include "pico/stdlib.h" //subconjunto central de bibliotecas do SDK Pico
#include "hardware/pwm.h" //biblioteca para controlar o hardware de PWM

#define PWM_LED 12 //pino do LED conectado a GPIO como PWM

//rotina principal
int main()
{
    stdio_init_all(); //inicializa o sistema padrão de I/O

    gpio_set_function(PWM_LED, GPIO_FUNC_PWM); //habilitar o pino GPIO como PWM

    uint slice = pwm_gpio_to_slice_num(PWM_LED); //obter o canal (slice) PWM da GPIO

    pwm_set_clkdiv(slice, 2.0); //define o divisor de clock do PWM

    pwm_set_wrap(slice, 2000); //definir o valor de wrap – valor máximo do contador PWM

    pwm_set_gpio_level(PWM_LED, 1500); //definir o ciclo de trabalho (duty cycle) do pwm

    pwm_set_enabled(slice, true); //habilitar o pwm no slice correspondente

    //loop principal
    while (true) {

        printf("slice:%d\n",slice);
        sleep_ms(1000);
    }
}
```

Cod: pwm_01

Duty Cycle (D_C).

$$D_C = \frac{t_p}{T_{PWM}} \times 100$$

$$D_C = \frac{1500}{2000} \times 100 = 75\%$$

t_p-> Tempo de pulso

Configuração do módulo PWM – Exemplo 1

Código fonte.

```c
#include <stdio.h> //biblioteca padrão da linguagem C
#include "pico/stdlib.h" //subconjunto central de bibliotecas do SDK Pico
#include "hardware/pwm.h" //biblioteca para controlar o hardware de PWM

#define PWM_LED 12 //pino do LED conectado a GPIO como PWM

//rotina principal
int main()
{
    stdio_init_all(); //inicializa o sistema padrão de I/O

    gpio_set_function(PWM_LED, GPIO_FUNC_PWM); //habilitar o pino GPIO como PWM

    uint slice = pwm_gpio_to_slice_num(PWM_LED); //obter o canal (slice) PWM da GPIO

    pwm_set_clkdiv(slice, 2.0); //define o divisor de clock do PWM

    pwm_set_wrap(slice, 2000); //definir o valor de wrap – valor máximo do contador PWM

    pwm_set_gpio_level(PWM_LED, 1500); //definir o ciclo de trabalho (duty cycle) do pwm

    pwm_set_enabled(slice, true); //habilitar o pwm no slice correspondente

    //loop principal
    while (true) {

        printf("slice:%d\n",slice);
        sleep_ms(1000);
    }
}
```

Cod: pwm_01

Tensão Média (V_{med}).

$$V_{med} = V_{pico} \times D_C$$

$$V_{med} = 3{,}3 \times 0{,}75 = 2{,}475\ V$$

Configuração do módulo PWM – Exemplo 1

Código fonte.

```c
#include <stdio.h> //biblioteca padrão da linguagem C
#include "pico/stdlib.h" //subconjunto central de bibliotecas do SDK Pico
#include "hardware/pwm.h" //biblioteca para controlar o hardware de PWM

#define PWM_LED 12 //pino do LED conectado a GPIO como PWM

//rotina principal
int main()
{
    stdio_init_all(); //inicializa o sistema padrão de I/O

    gpio_set_function(PWM_LED, GPIO_FUNC_PWM); //habilitar o pino GPIO como PWM

    uint slice = pwm_gpio_to_slice_num(PWM_LED); //obter o canal (slice) PWM da GPIO

    pwm_set_clkdiv(slice, 2.0); //define o divisor de clock do PWM

    pwm_set_wrap(slice, 2000); //definir o valor de wrap – valor máximo do contador PWM

    pwm_set_gpio_level(PWM_LED, 1500); //definir o ciclo de trabalho (duty cycle) do pwm

    pwm_set_enabled(slice, true); //habilitar o pwm no slice correspondente

    //loop principal
    while (true) {

        printf("slice:%d\n",slice);
        sleep_ms(1000);
    }
}
```

Cod: pwm_01

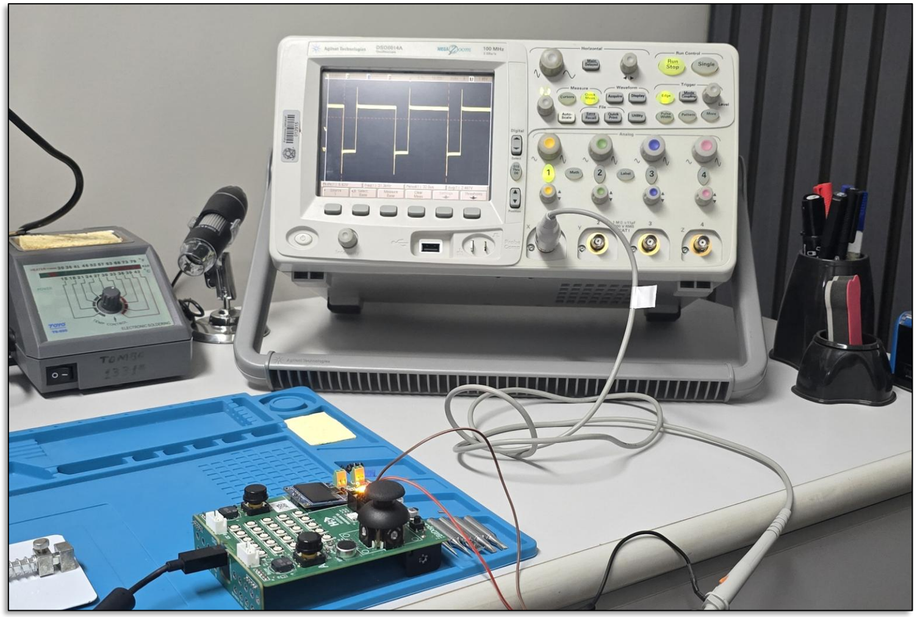
Valor Eficaz (V_{RMS}).

$$V_{RMS} = V_{pico} \times \sqrt{D_C}$$

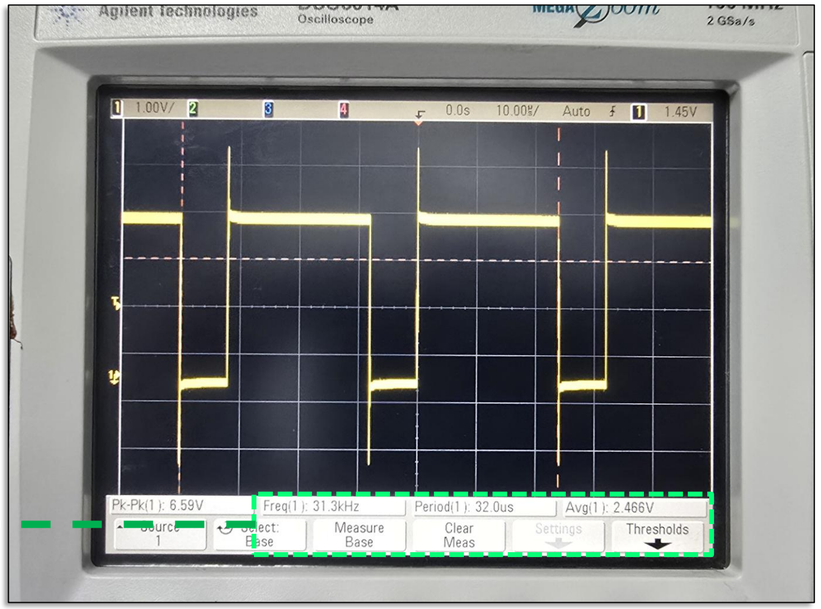
$$V_{RMS} = 3{,}3 \times \sqrt{0{,}75} = 2{,}86\ V$$

Configuração do módulo PWM – Exemplo 1

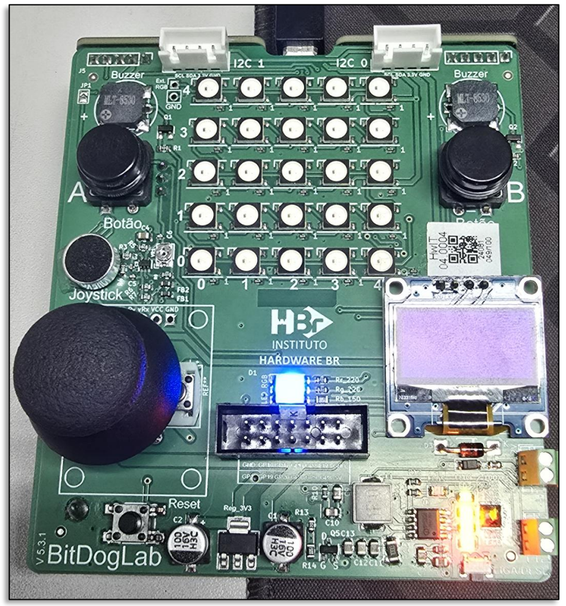
Experimento



Osciloscópio - *Duty Cycle* = 75%



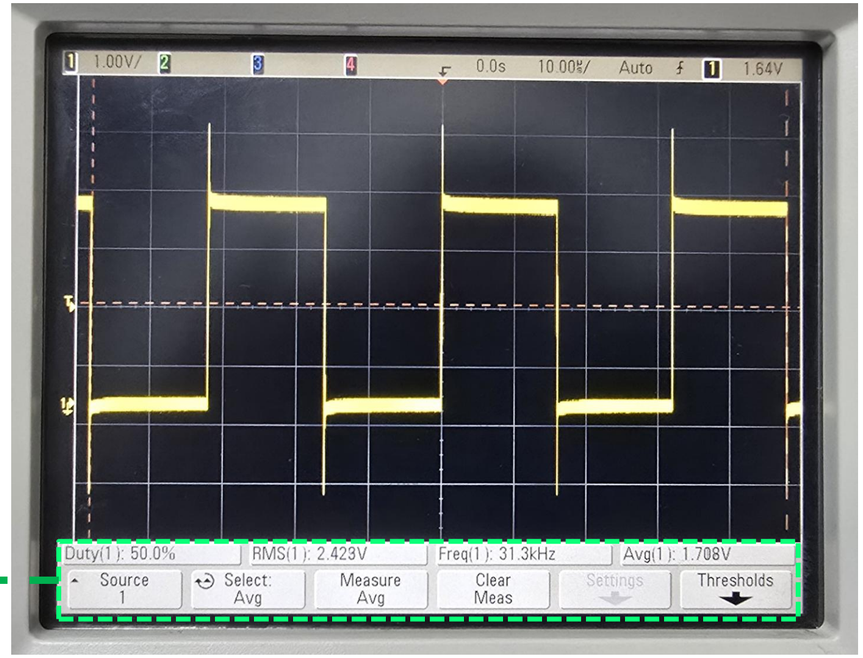
BitDogLab



Frequência = 31,3kHz
Período = 32μ s
Vmed = 2,466 V
Vrms = 2,89 V

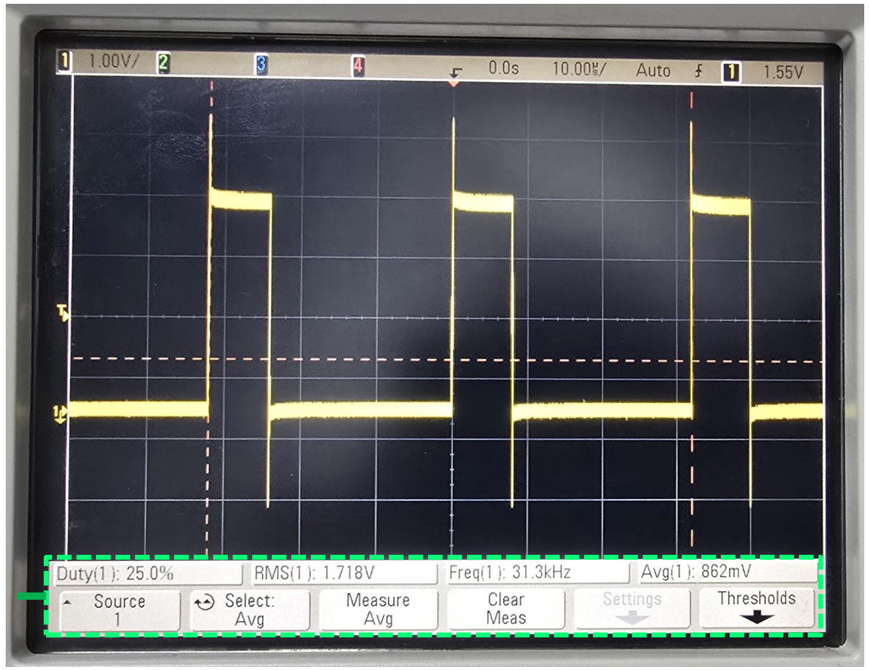
Configuração do módulo PWM – Exemplo 1

Duty Cycle = 50%



Frequência = 31,3kHz
Período = 32μ s
Vmed = 1,798 V
Vrms = 2,89 V

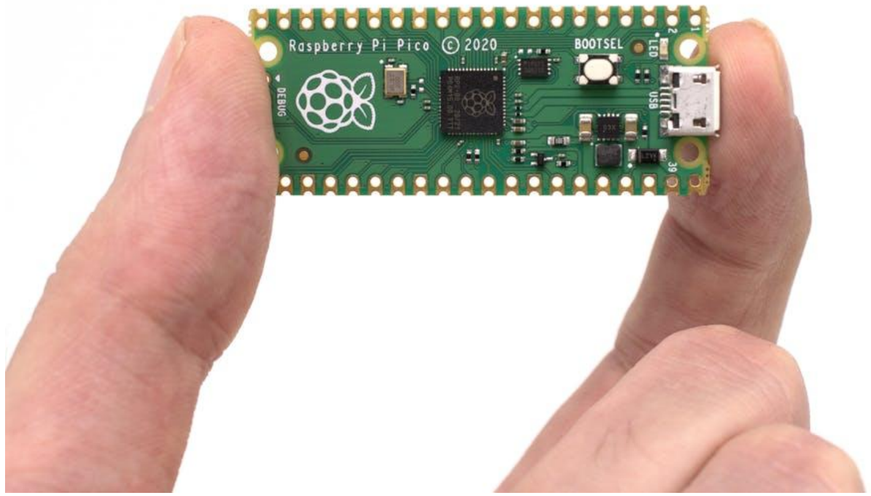
Duty Cycle = 25%



Frequência = 31,3kHz
Período = 32μ s
Vmed = 0,862 V
Vrms = 1,718 V

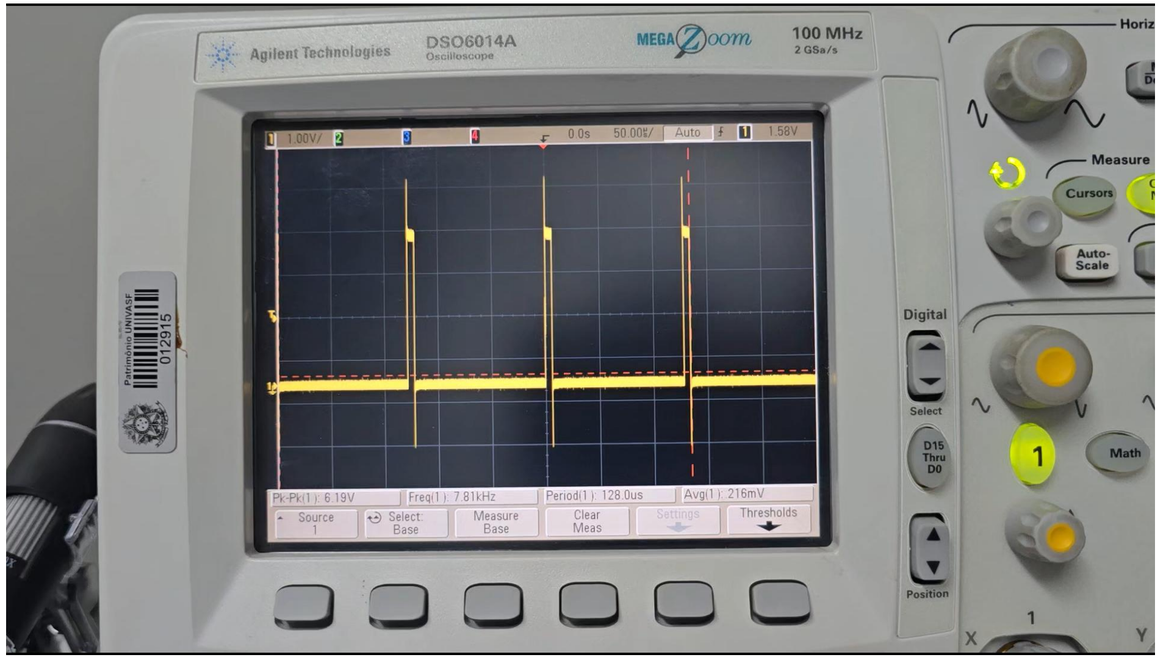
Vamos para o microcontrolador!

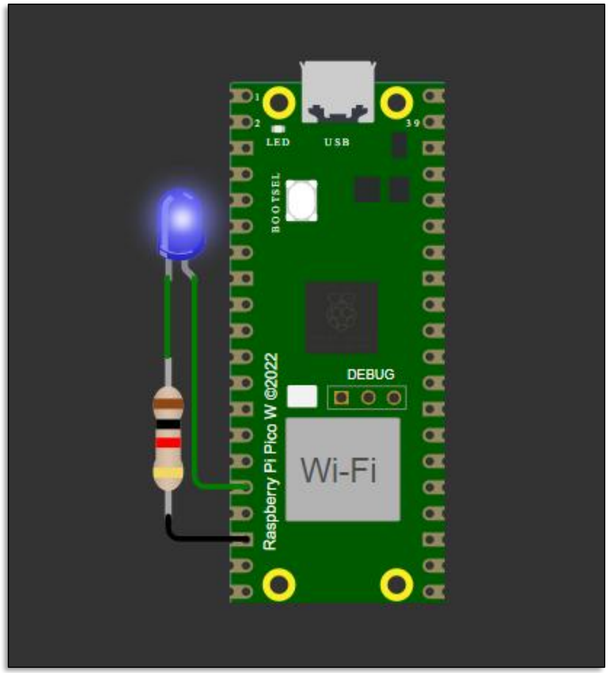
Exemplo 2



Controle do Duty Cycle – Exemplo 2

Objetivo: Realizar o controle do *Duty Cycle* no loop principal. Com isto, variar o nível de intensidade de iluminação de um LED.





GitHub

Controle do Duty Cycle – Exemplo 2

Código fonte – parte 1/2.

```c
#include <stdio.h> //biblioteca padrão da linguagem C
#include "pico/stdlib.h" //subconjunto central de bibliotecas do SDK Pico
#include "hardware/pwm.h" //biblioteca para controlar o hardware de PWM

#define PWM_LED 12 //pino do LED conectado a GPIO como PWM
const uint16_t WRAP_PERIOD = 4000; //valor máximo do contador - WRAP
const float PWM_DIVISER = 4.0; //divisor do clock para o PWM
const uint16_t LED_STEP = 200; //passo de incremento/decremento para o duty cycle do LED
uint16_t led_level = 200; //nível inicial do pwm (duty cycle)

//função para configurar o módulo PWM
void pwm_setup()
{
    gpio_set_function(PWM_LED, GPIO_FUNC_PWM); //habilitar o pino GPIO como PWM

    uint slice = pwm_gpio_to_slice_num(PWM_LED); //obter o canal PWM da GPIO

    pwm_set_clkdiv(slice, PWM_DIVISER); //define o divisor de clock do PWM

    pwm_set_wrap(slice, WRAP_PERIOD); //definir o valor de wrap

    pwm_set_gpio_level(PWM_LED, 100); //definir o cico de trabalho (duty cycle) do pwm

    pwm_set_enabled(slice, true); //habilita o pwm no slice correspondente
}
```

$$f_{PWM} = \frac{f_{clock}}{(d_i + \frac{d_f}{16}) \times (\text{wrap}+1)}$$

$$f_{PWM} = \frac{125 \times 10^6}{(4 + \frac{0}{16}) \times 4 \times 10^3}$$

Cod: pwm_config_01

Controle do Duty Cycle – Exemplo 2

Código fonte – parte 1/2.

```c
#include <stdio.h> //biblioteca padrão da linguagem C
#include "pico/stdlib.h" //subconjunto central de bibliotecas do SDK Pico
#include "hardware/pwm.h" //biblioteca para controlar o hardware de PWM

#define PWM_LED 12 //pino do LED conectado a GPIO como PWM
const uint16_t WRAP_PERIOD = 4000; //valor máximo do contador - WRAP
const float PWM_DIVISER = 4.0; //divisor do clock para o PWM
const uint16_t LED_STEP = 200; //passo de incremento/decremento para o duty cycle do LED
uint16_t led_level = 200; //nível inicial do pwm (duty cycle)

//função para configurar o módulo PWM
void pwm_setup()
{
    gpio_set_function(PWM_LED, GPIO_FUNC_PWM); //habilitar o pino GPIO como PWM

    uint slice = pwm_gpio_to_slice_num(PWM_LED); //obter o canal PWM da GPIO

    pwm_set_clkdiv(slice, PWM_DIVISER); //define o divisor de clock do PWM

    pwm_set_wrap(slice, WRAP_PERIOD); //definir o valor de wrap

    pwm_set_gpio_level(PWM_LED, 100); //definir o cico de trabalho (duty cycle) do pwm

    pwm_set_enabled(slice, true); //habilita o pwm no slice correspondente
}
```

$$f_{PWM} = 7.812{,}5\text{Hz}$$

$$T_{PWM} = \frac{1}{f_{PWM}} = 128\mu s$$

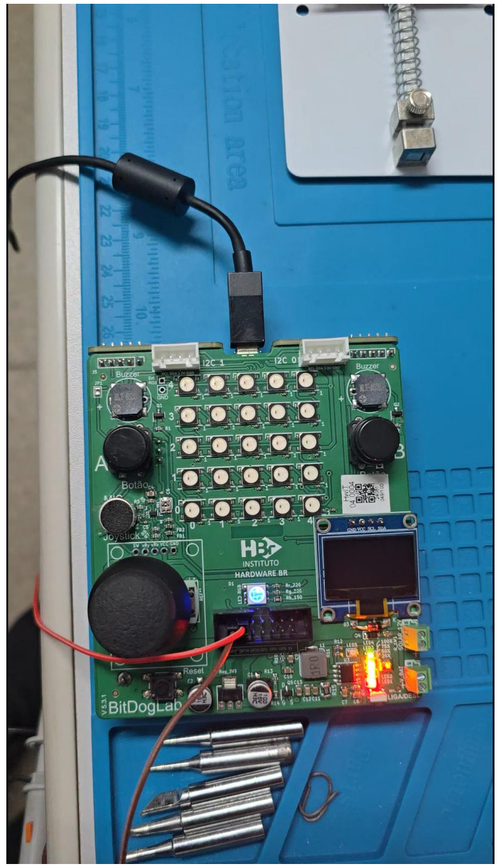
Cod: freq_03

Controle do Duty Cycle – Exemplo 2

Código fonte – parte 2/2.

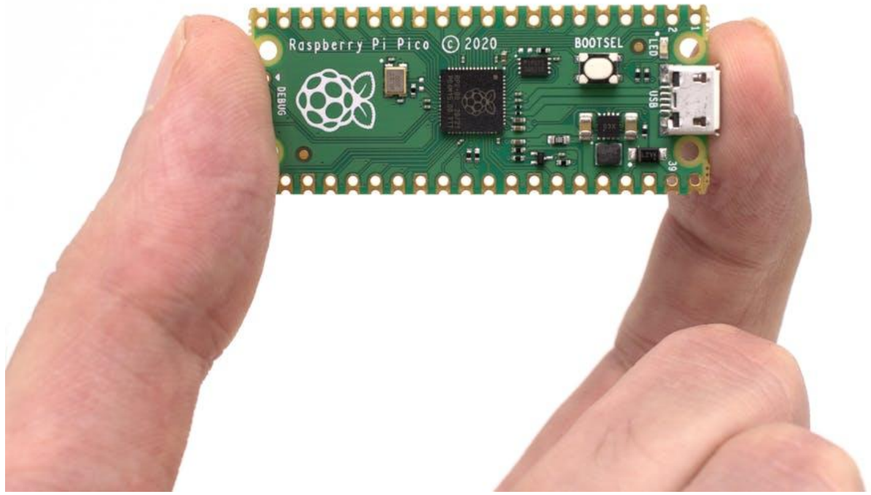
```c
//função principal
int main()
{
    stdio_init_all(); //inicializa o sistema padrão de I/O

    pwm_setup(); //configura o PWM

    uint up_down = 1; //variável para controlar se o nível do LED aumenta ou diminui

    //loop principal
    while (true) {

        printf("Ciclo ativo:%d\n", led_level);//imprimir ciclo ativo do PWM - valor máximo é 2000

        pwm_set_gpio_level(PWM_LED, led_level); //define o nível atual do PWM (duty cycle)

        sleep_ms(1000); // Atraso de 1 segundo

        if (up_down) //condição para elevação/redução do ciclo de trabalho.
        {
            led_level += LED_STEP; // Incrementa o nível do LED
            if (led_level >= WRAP_PERIOD)
                up_down = 0; // Muda direção para diminuir quando atingir o período máximo
        }
        else
        {
            led_level -= LED_STEP; // Decrementa o nível do LED
            if (led_level <= LED_STEP)
                up_down = 1; // Muda direção para aumentar quando atingir o mínimo
        }

    }
}
```

BitDogLab



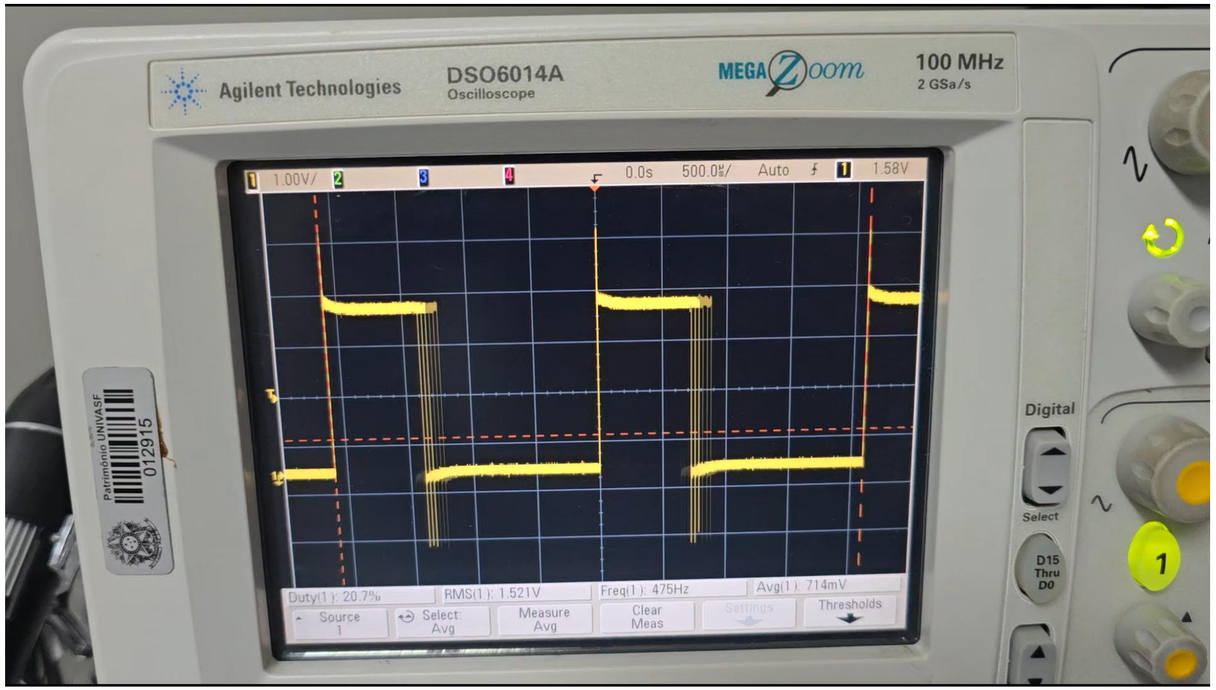
Vamos para o microcontrolador!

Exemplo 3
Atividade Individual

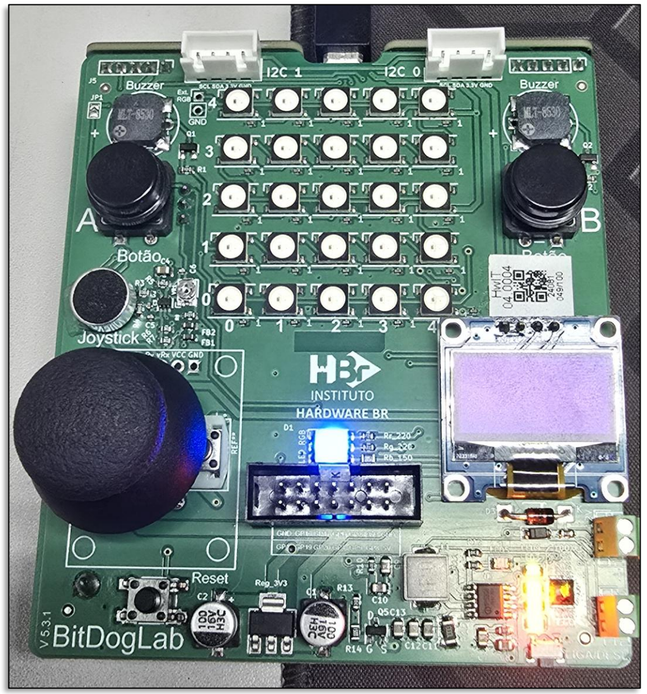


PWM com Interrupção – Exemplo 3

Objetivo: Realizar o controle do *Duty Cycle* através de interrupção. Com isto, variar o nível de intensidade de iluminação de um LED.



BitDogLab



PWM com Interrupção – Exemplo 3

Código fonte – parte 1/2.

```c
#include <stdio.h> //biblioteca padrão da linguagem C
#include "pico/stdlib.h" //subconjunto central de bibliotecas do SDK Pico
#include "pico/time.h" //biblioteca para gerenciamento de tempo
#include "hardware/irq.h" //biblioteca para gerenciamento de interrupções
#include "hardware/pwm.h" //biblioteca para controlar o hardware de PWM

#define LEDPin 12 //pino do LED conectado a GPIO como PWM

//tratamento da interrupção do PWM
void wrapHandler(){
    static int fade = 0; //nível de iluminação
    static bool rise = true; //flag para elevar ou reduzir a iluminação
    pwm_clear_irq(pwm_gpio_to_slice_num(LEDPin)); //resetar o flag de interrupção

    if(rise){ //caso a iluminação seja elevada
        fade++; //aumenta o nível de brilho
        if(fade > 255){ //caso o fade seja maior que 255
            fade = 255; //iguala fade a 255
            rise = false; //muda o flag rise para redução do nível de iluminação
        }
    }
    else{ //caso a iluminação seja reduzida
        fade--; //reduz o nível de brilho
        if(fade < 0){ //Icaso o fade seja menor que 0
            fade = 0; //iguala fade a 0
            rise = true; //muda o flag rise para elevação no nível de iluminação
        }
    }

    pwm_set_gpio_level(LEDPin, fade * fade); //define o ciclo ativo (Ton) de forma quadrática, para acentuar a variação de luminosidade.
}
```

Cod: pwm_irq

PWM com Interrupção – Exemplo 3

Código fonte – parte 2/2.

```c
//Configuração do PWM com interrupção
uint pwm_setup_irq(){

    gpio_set_function(LEDPin, GPIO_FUNC_PWM); //habilitar o pino GPIO como PWM
    uint sliceNum = pwm_gpio_to_slice_num(LEDPin); //obter o canal PWM da GPIO

    pwm_clear_irq(sliceNum); //resetar o flag de interrupção para o slice
    pwm_set_irq_enabled(sliceNum, true); //habilitar a interrupção de PWM para um dado slice
    irq_set_exclusive_handler(PWM_IRQ_WRAP, wrapHandler); //Definir um tipo de interrupção.
    // Esta interrupção (PWM_IRQ_WRAP) é gerada quando um contador de um slice atinge seu valor de wrap
    irq_set_enabled(PWM_IRQ_WRAP, true); //Habilitar ou desabilitar uma interrupção específica

    pwm_config config = pwm_get_default_config(); //obtem a configuração padrão para o PWM
    pwm_config_set_clkdiv(&config, 4.f); //define o divisor de clock do PWM
    pwm_init(sliceNum, &config, true); //inicializa o PWM com as configurações do objeto

    return sliceNum;
}

int main(){

    uint sliceNum = pwm_setup_irq(); //função que inicializa o PWM com interrupção por wrap

    while(1){

        printf("Interrupção do PWM ativa");
        pwm_set_irq_enabled(sliceNum, true); //habilita a interrupção
        sleep_ms(5000);
        printf("Interrupção do  PWM desativada"); //desabilita a interrupção
        pwm_set_irq_enabled(sliceNum, false);
        sleep_ms(5000);

    }
}
```

$$f_{PWM} = \frac{125 \times 10^6}{(4 + \frac{0}{16}) \times 65.535}$$

$$f_{PWM} = 476{,}84\text{Hz}$$

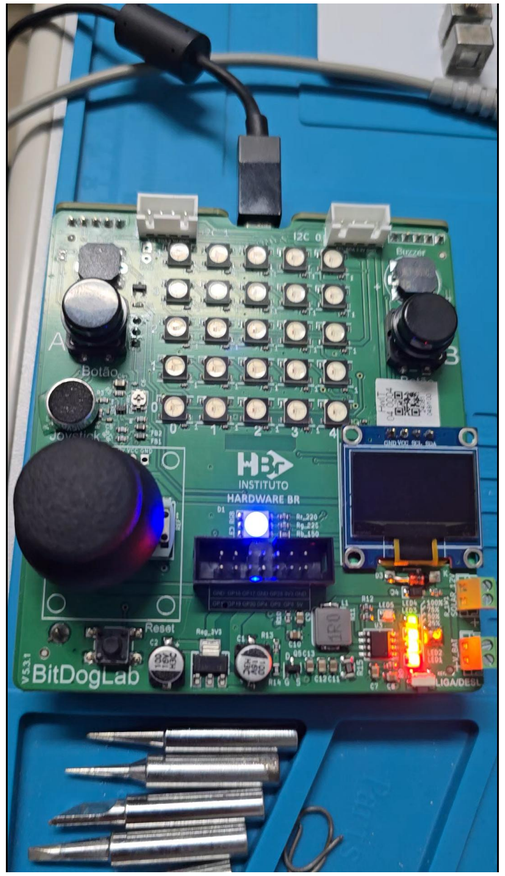
$$T_{PWM} = 2{,}1ms$$

PWM com Interrupção – Exemplo 3

Código fonte – parte 2/2.

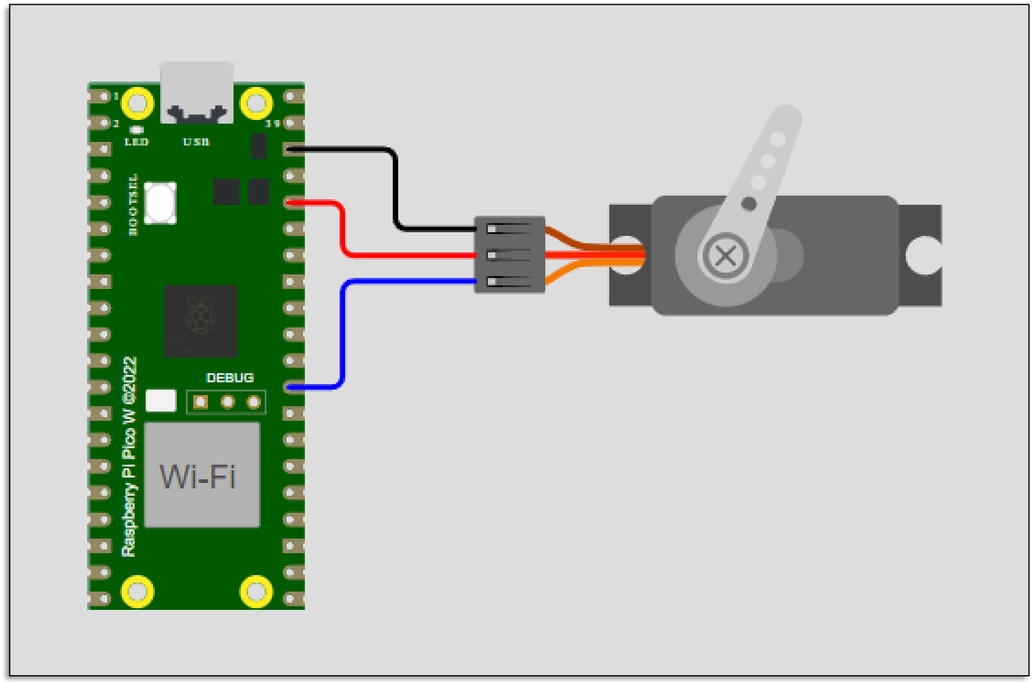
```c
//Configuração do PWM com interrupção
uint pwm_setup_irq(){

    gpio_set_function(LEDPin, GPIO_FUNC_PWM); //habilitar o pino GPIO como PWM
    uint sliceNum = pwm_gpio_to_slice_num(LEDPin); //obter o canal PWM da GPIO

    pwm_clear_irq(sliceNum); //resetar o flag de interrupção para o slice
    pwm_set_irq_enabled(sliceNum, true); //habilitar a interrupção de PWM para um dado slice
    irq_set_exclusive_handler(PWM_IRQ_WRAP, wrapHandler); //Definir um tipo de interrupção.
    // Esta interrupção (PWM_IRQ_WRAP) é gerada quando um contador de um slice atinge seu valor de wrap
    irq_set_enabled(PWM_IRQ_WRAP, true); //Habilitar ou desabilitar uma interrupção específica

    pwm_config config = pwm_get_default_config(); //obtem a configuração padrão para o PWM
    pwm_config_set_clkdiv(&config, 4.f); //define o divisor de clock do PWM
    pwm_init(sliceNum, &config, true); //inicializa o PWM com as configurações do objeto

    return sliceNum;
}

int main(){

    uint sliceNum = pwm_setup_irq(); //função que inicializa o PWM com interrupção por wrap

    while(1){

        printf("Interrupção do PWM ativa");
        pwm_set_irq_enabled(sliceNum, true); //habilita a interrupção
        sleep_ms(5000);
        printf("Interrupção do  PWM desativada"); //desabilita a interrupção
        pwm_set_irq_enabled(sliceNum, false);
        sleep_ms(5000);

    }
}
```

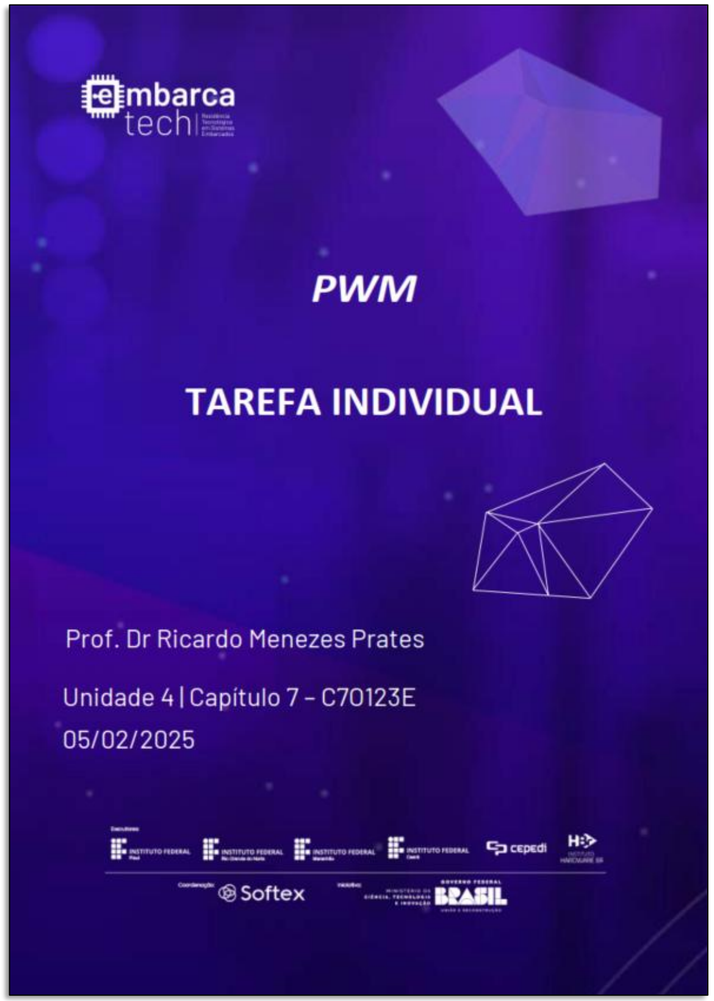
BitDogLab



Controle de servomotor – Atividade

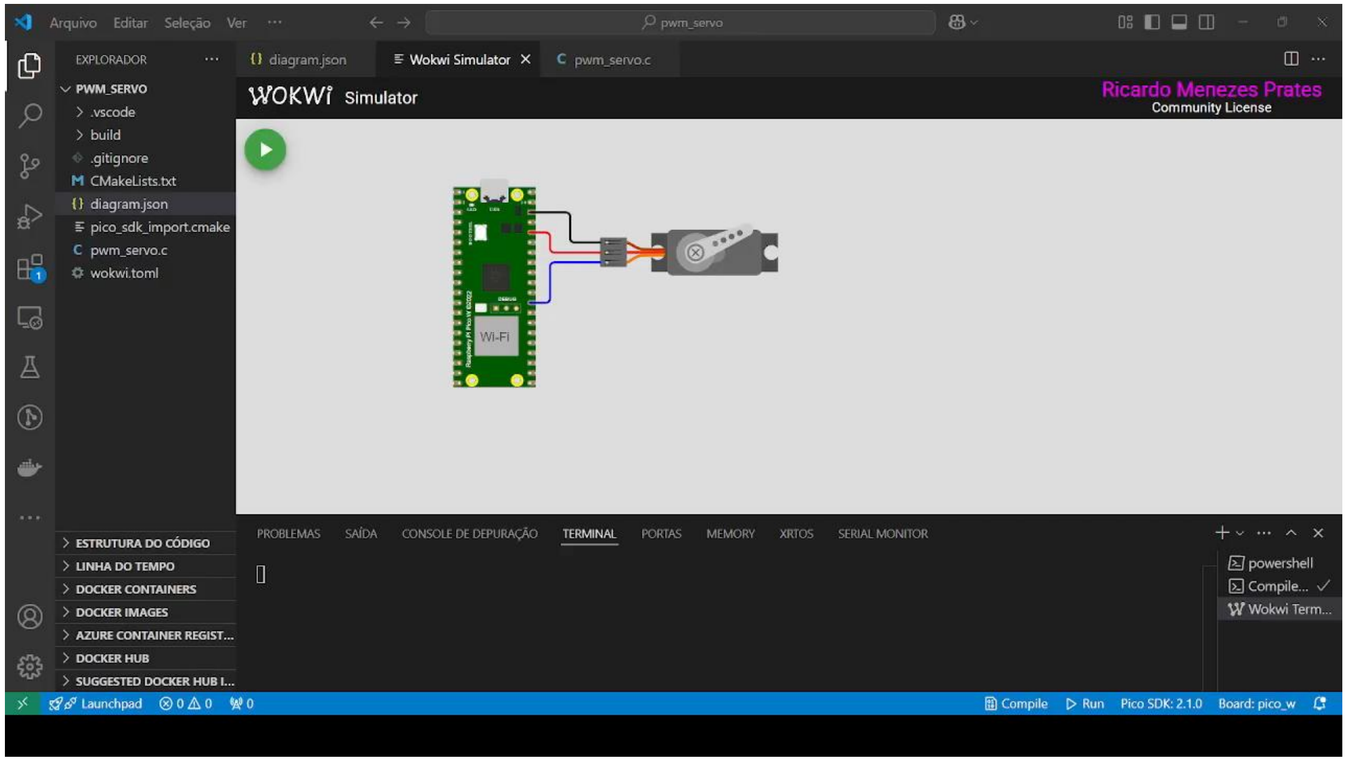
Exemplo: Veja o documento da atividade:





Controle de servomotor – Atividade

Exemplo: Veja o documento da atividade:

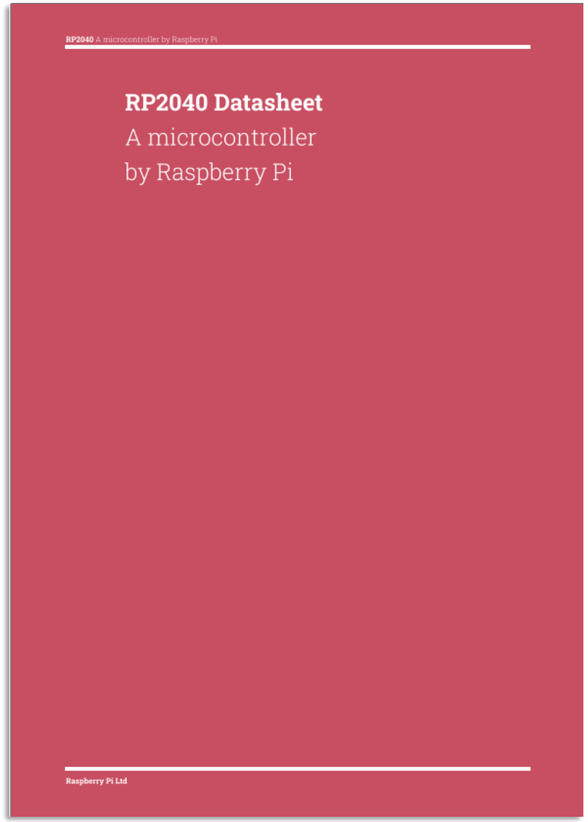


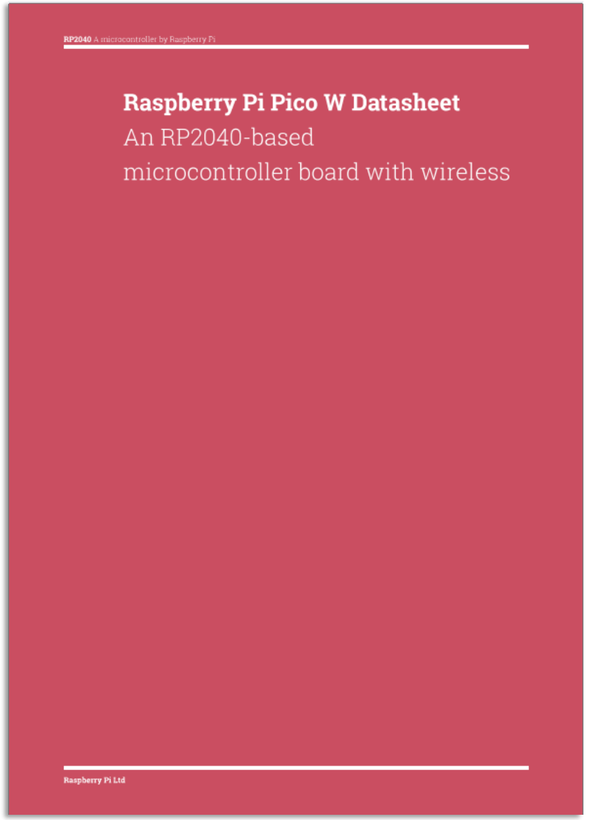
Conclusões

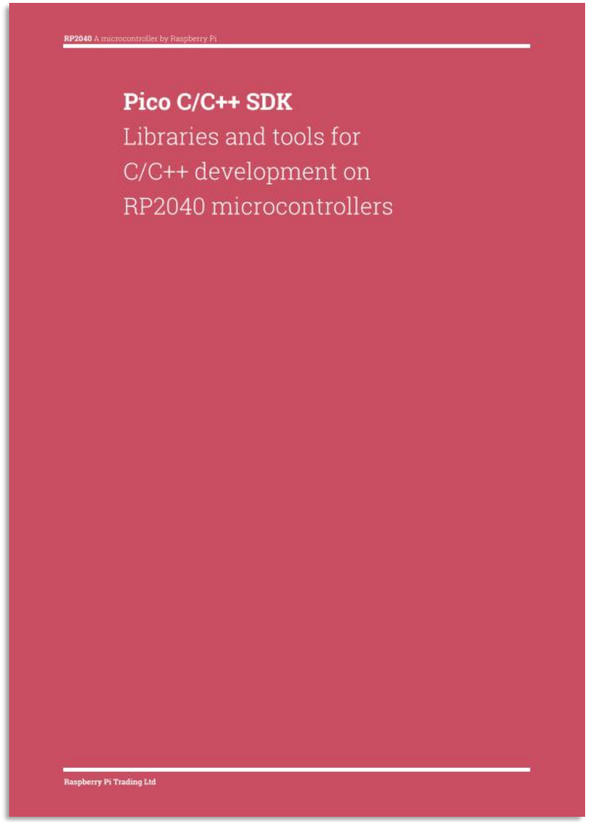
Conclusões

- O microcontrolador RP2040 conta com um conjunto de **geradores de PWM** (slices), permitindo 16 saídas simultâneas.
- Além de gerar sinais de PWM, estes geradores podem ser usados para outras finalidades, a exemplo de medição de período ou ciclo ativo de sinais de entrada.
- Qualquer pino GPIO pode ser configurado como uma saída PWM, facilitando o roteamento dos sinais conforme a necessidade do projetista.
- Este periférico é customizável para uma ampla faixa de frequências e ainda conta com interrupções para o desacoplamento das suas rotinas ao bloco de códigos principal.
- O PWM pode ser empregado em diferentes tipos de aplicação, a exemplo do controle de motores e de intensidade luminosa de LEDs, se tornando um recurso bastante atrativo para projetistas de sistemas embarcados.

Bibliografia Recomendada







embarca tech | Residência Tecnológica em Sistemas Embarcados

Modulação por Largura de Pulso (PWM)

Unidade 4 | Capítulo 7 – Aula Síncrona (06/02/2025)

Executores: INSTITUTO FEDERAL Piauí, INSTITUTO FEDERAL Rio Grande do Norte, INSTITUTO FEDERAL Maranhão, INSTITUTO FEDERAL Ceará, cepedi, INSTITUTO HARDWARE BR

Coordenação: Softex

Iniciativa: MINISTÉRIO DA CIÊNCIA, TECNOLOGIA E INOVAÇÃO, GOVERNO FEDERAL BRASIL UNIÃO E RECONSTRUÇÃO